UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros de Telecomunicación
Grupo de Tratamiento de Imágenes

Proyectos en Ingeniería de Datos y Sistemas (PIDS)

Proyecto 2

Sistema de detección de vehículos y conducción autónoma

MEMORIA DEL PROYECTO FASES 1, 2 Y 3

Grupo	2
Autores	Javier Saguar Alejandro Cuevas Mónica Fernández Pedro José Orrego Daniel Naval
Entrega	Entrega 3 · Bloques 1, 2 y 3
Fecha de entrega	15/11/2026

Reparto del trabajo

Integrante	Tareas del Bloque 1	Tareas del Bloque 2	Tareas del Bloque 3
Javier Saguar	Lote A (vídeos 01-02): extracción, anotación y revisión cruzada. Fichas de los vídeos, criterios de anotación y depuración del dataset. Aumento de datos sobre el script de Alejandro, que mejoró y ejecutó. Redacción de la memoria.	Cadena de entrenamientos (siete arquitecturas y tres tasas de aprendizaje), evaluación final en test y comprobación con 40 épocas. Gráficas comparativas de la web y de la memoria.	Demo del circuito: desplazamientos, puntería, disparo, ataques en marcha y robot camuflado. Demo con el detector en la GPU y la CPU de respaldo, kit del rival móvil, interfaz del mando y memoria del Bloque 3.
Alejandro Cuevas	Lote B (vídeos 03-04): extracción, anotación y revisión cruzada. Análisis exploratorio de los datos (EDA) y script de entrenamiento sobre el que se integró el aumento. Redacción de la memoria.	Adaptación del código de entrenamiento y test, propuestas de mejora del entrenamiento y redacción de la memoria del Bloque 2.	Seguimiento de rivales en movimiento, barrido continuo de la torreta, memoria de robots atacados y vigilancia del recorrido.
Mónica Fernández	Lote C (vídeos 05-06): extracción, anotación y revisión cruzada. Calibración entre anotadores y revisión de la memoria.	Correcciones memoria Bloque 1.	Diseño del boceto del circuito.
Pedro José Orrego	Lote D (vídeos 07-08): extracción, anotación y revisión cruzada. Calibración, selección de figuras del EDA y diseño de la web de figuras interactivas (GitHub Pages).	Web interactiva del Bloque 2 y navegación común con la del Bloque 1 (GitHub Pages).	Conexión y preparación del boceto de la demo.
Daniel Naval	Lote E (vídeos 09-10): extracción, anotación y revisión cruzada. Calibración y redacción de la memoria.		Ayuda en el montaje del circuito.

Tabla 1. Reparto del trabajo de los Bloques 1, 2 y 3 entre los cinco integrantes.

Índice

Reparto del trabajo 2
Índice 3
1. Introducción 4
2. Recolección de datos 4
3. Anotación de datos 5
4. Análisis exploratorio de los datos 6
5. Aumento de datos 11
6. Depuración de datos 13
7. El pipeline de entrenamiento 14
8. Modelos y transfer learning 15
9. Hiperparámetros 16
10. Métricas 18
11. Experimentos 19
12. Análisis cualitativo 24
13. Integración del detector en el robot 26
14. El circuito 26
15. Detección, puntería y disparo 27
16. Robot camuflado y robots aliados 28
17. Control manual del robot 29
18. Pruebas y estado 31
19. Conclusiones 32
Referencias 33
Anexos 35
Respuesta a la revisión anterior 46

1. Introducción

1.1. Contexto y motivación

El proyecto consiste en desarrollar un detector de robots RoboMaster rivales a partir de las imágenes captadas por la cámara de un robot móvil. Esta tarea presenta varias dificultades, ya que el movimiento del robot, los cambios de tamaño aparente, las condiciones de iluminación y la presencia de obstáculos pueden afectar a la localización de los robots. La detección obtenida servirá como base para desarrollar posteriormente una estrategia de apuntado y actuación.

Los robots que hay que detectar son RoboMaster EP de DJI [19], el mismo modelo que lleva la cámara. Es un robot terrestre educativo con chasis de ruedas mecanum, que le permiten desplazarse en cualquier dirección, y una torreta (gimbal) que orienta la cámara y el lanzador (blaster). Las placas de impacto del chasis y la torreta llevan indicadores LED de color (Figura 1). Esa silueta, con sus luces, es lo que el detector tiene que reconocer desde cualquier ángulo y distancia.



Figura 1. Partes de un RoboMaster EP en un fotograma de V03. Las antenas quedan fuera de la caja (Tabla 4).

1.2. Objetivos del proyecto

El trabajo se divide en tres bloques. El primero se centra en la creación y el análisis del conjunto de datos, el segundo en el entrenamiento y ajuste del detector, y el tercero en su integración con el robot. En este primer bloque se obtuvieron 808 imágenes originales anotadas, de las cuales finalmente se seleccionaron 796 tras revisar las etiquetas y comprobar su calidad.

2. Recolección de datos

2.1. Extracción de imágenes

La cámara del robot graba a 1280 × 720 píxeles y unos 30 fps, así que dos fotogramas seguidos son casi iguales. Por eso se extrae uno de cada 24 fotogramas, una imagen cada 0,79 segundos. Con los 19 508 fotogramas de los diez vídeos (643,8 s de grabación) se obtienen 808 imágenes: cada vídeo aporta sus fotogramas divididos entre 24, redondeando hacia abajo (Tabla 2).

Vídeo	Escenario	Duración s	Frames	Imágenes
V01	Vestíbulo ETSIT	65,4	1982	82
V02	Aula A-201.L	59,9	1815	75
V03	Aula A-201.L	60,2	1823	75

Vídeo	Escenario	Duración s	Frames	Imágenes
V04	Exterior edificio A	66,1	2003	83
V05	Pasillo A-201.L	59,6	1807	75
V06	Vestíbulo ETSIT (mesas)	74,1	2244	93
V07	Vestíbulo ETSIT	61,2	1855	77
V08	Vestíbulo ETSIT (mesas)	81,2	2460	102
V09	Exterior edificio A	56,2	1704	71
V10	Aula A-201.L	59,9	1815	75
Total	1280 × 720 a 30,30 fps	643,8	19 508	808

Tabla 2. Grabaciones propias y número exacto de imágenes extraídas.

El intervalo se eligió por el compromiso entre redundancia y cantidad de datos. Para comprobarlo, se repitió la extracción con otros intervalos y se midió cuánto cambia cada imagen respecto a la anterior, con la misma medida del EDA: la diferencia media de gris entre miniaturas, con la que una pareja por debajo de 5 se considera casi duplicada (Tabla 3).

Intervalo (fotogramas)	Segundos	Imágenes	Casi duplicadas	Diferencia mediana	Robots que anotar
6	0,20	3248	11,1 %	15,0	≈ 5 710
12	0,40	1622	4,6 %	22,6	≈ 2 850
24	0,79	808	1,4 %	29,2	1 420
48	1,58	400	0,3 %	32,8	≈ 700
96	3,17	196	0,0 %	36,9	≈ 340

Tabla 3. Redundancia de las imágenes extraídas según el intervalo, sobre los diez vídeos. Los robots se estiman con la media de 1,76 por imagen.

Con 12 fotogramas, el 4,6 % de las imágenes son casi duplicadas y habría que anotar el doble de robots para añadir poca información (diferencia mediana de 22,6). Con 24 la redundancia cae al 1,4 %, y duplicar el intervalo apenas cambia la diferencia (de 29,2 a 32,8) pero deja la mitad de ejemplos, 400 imágenes. Extraer más fotogramas tampoco aumentaría la variedad: la imagen más parecida a cada una ya es del mismo vídeo en el 74 % de los casos (apartado 4.2). La variedad depende de grabar escenarios distintos.

3. Anotación de datos

Se utilizó labelImg [1], que guarda las cajas en el formato XML de PASCAL VOC [20], con una única clase robot. Cada integrante anotó su lote y otro revisó las cajas. Se conservaron imágenes negativas con XML vacío para que el entrenamiento no las ignore.

Caso	Criterio operativo
Robot rival	Incluir chasis, ruedas, gimbal y cañón; excluir antenas.
Elementos ajenos	No anotar el cañón propio, reflejos ni robots en pantallas.
Oclusión y borde	Anotar si es reconocible y se ve al menos un 30 %; caja sobre la parte visible.
Difficult	Marcar si tiene menos de 30 px de ancho, menos de la mitad visible o está muy movido.
Robot muy pequeño	No anotar por debajo de 10 px de lado; conservar criterios en el aumento.
Negativo	Guardar imagen y XML sin objetos.

Tabla 4. Criterios comunes aplicados a los lotes.

Como se observa en la figura siguiente, el criterio distingue cajas normales y difficult sin crear una segunda clase. Una frontera superior distinta alteraría tamaños y centros, por lo que se excluyen antenas y se ajusta el resto del robot de manera consistente.



Figura 2. Anotaciones de interior y exterior. Azul: normal; naranja: difficult.

Los cinco anotadores etiquetaron las mismas 20 imágenes de calibración. El IoU medio entre parejas fue 0,81; el borde superior presenta la mayor discrepancia, 7 px de mediana frente a 4–5 px en los otros bordes. Esto se debe a que la parte superior del robot es la más ambigua de anotar, debido a las antenas.

4. Análisis exploratorio de los datos

4.1. Inventario y equilibrio

El EDA se calcula sobre los 808 originales, antes de excluir imágenes. Los aumentos no se mezclan en estas estadísticas. Las cifras de la tabla y la figura permiten separar cantidad de imágenes, número de objetos y dificultad. El notebook del EDA adjunto [7] contiene más análisis (acuerdo entre anotadores, diversidad visual, duplicados y línea temporal).

Estadística	Valor
Secuencias / imágenes	10 / 808
Imágenes medias por secuencia	80,8

Estadística	Valor
Cajas / cajas medias por imagen	1420 / 1,76
Difficult	375 cajas · 26,4 %
Imágenes negativas	41 · 5,07 %
Área media de caja	33,627 miles de píxeles²
Área mínima / máxima	1,020 / 661,074 miles de píxeles²
Aspecto mediano ancho / alto	1,11

Tabla 5. Estadísticas de los originales anotados.

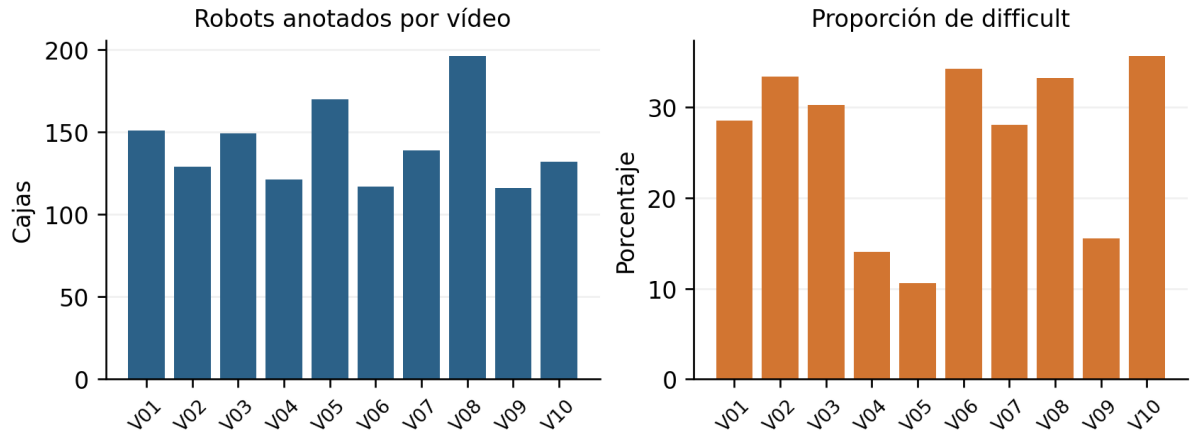


Figura 3. Recuento y dificultad por vídeo; sus diferencias motivan analizar cada secuencia.

No hemos intentado que todos los conjuntos sean exactamente iguales. Por ejemplo, V05 registra 170 robots, mientras que V06 contiene 117. Además, V06 incluye 22 ejemplos negativos, mientras que V08 no tiene ninguno y tiene 65 casos especialmente difíciles. Se conservan porque son situaciones reales, pero obligan a medir el AP de los difficult por separado (apartado 10): un AP global alto podría esconder que fallan.

4.2. Tamaño, forma e iluminación

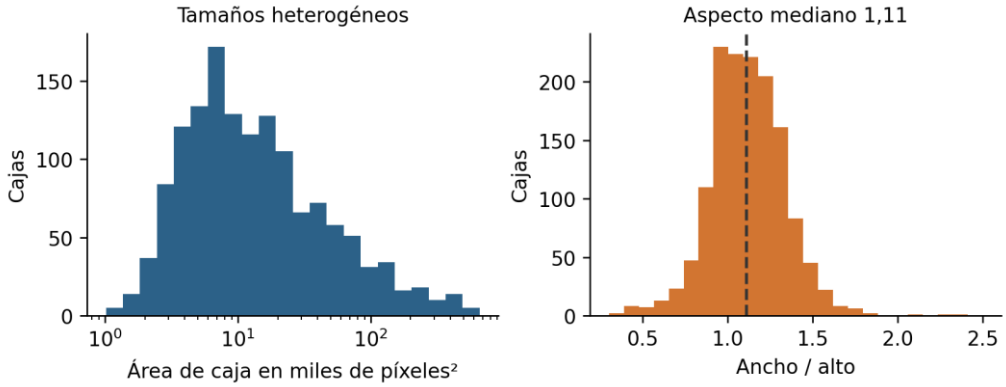


Figura 4. Área y forma de las cajas de los 1420 robots. El eje de área es logarítmico.

La distribución de la figura tiene una cola amplia de áreas, pero no hay cajas originales de menos de 30 px de ancho. La relación de aspecto mediana es 1,11. El análisis de cobertura de anclas obtiene un

99,6 % con IoU ≥0,5: no se cambia su configuración por ahora y se evita deformar las proporciones durante el aumento.

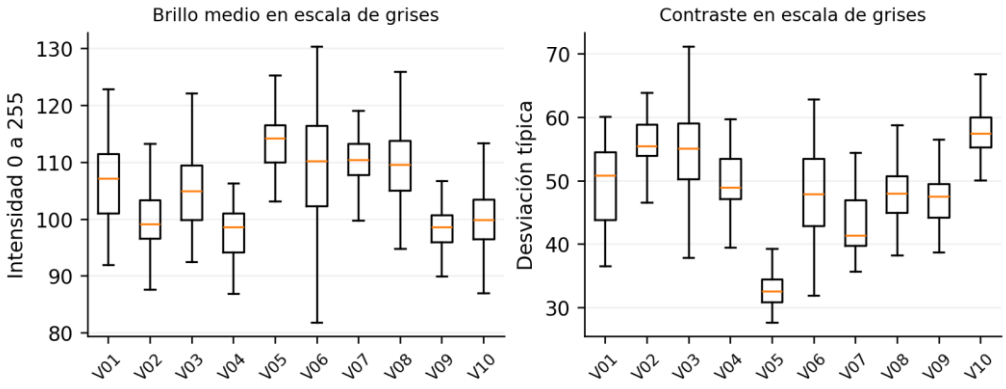


Figura 5. Brillo y contraste por secuencia, calculados en escala de grises.

Los canales RGB reflejan el color predominante de cada imagen y no solo las condiciones de iluminación, ya que también están influidos por elementos como el suelo o las paredes. Por ello, se complementan con medidas de intensidad y nitidez. Además, el tono azulado habitual en exteriores justifica aplicar ajustes de color moderados para no confundirlo con una iluminación deficiente. Se identificaron 52 imágenes poco nítidas para revisión visual; en cada vídeo, las menos nítidas tienen más cajas difficult (55 % frente a 11 %, Figura 22, Anexo C).

La miniatura más parecida a una imagen procede del mismo vídeo en el 74 % de los casos. Esa correlación aconseja separar grabaciones completas, evitando una validación artificialmente fácil con fotogramas vecinos.

4.3. Posición y evolución temporal

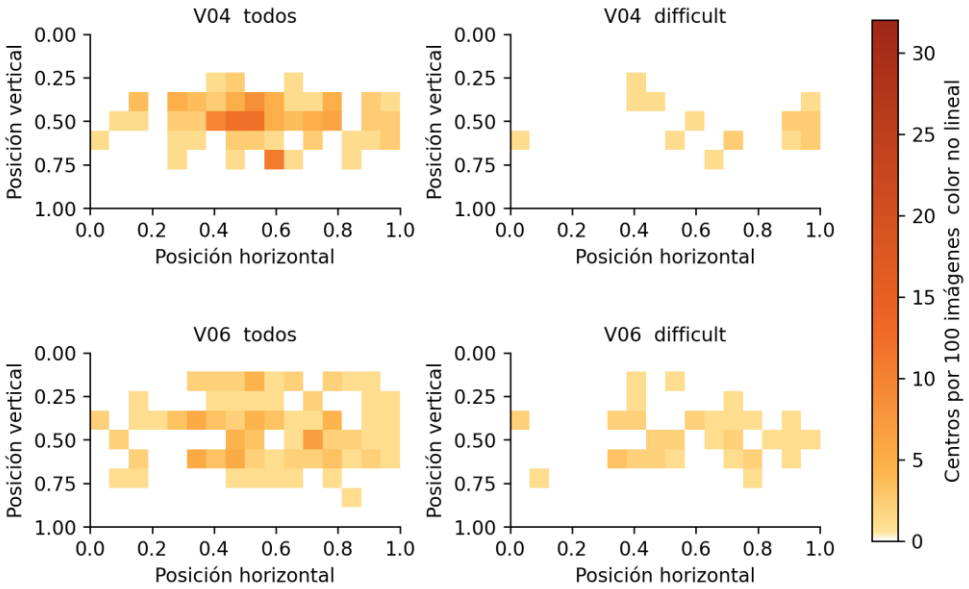


Figura 6. Centros de las cajas en V04 (exterior) y V06 (interior). Todos y difficult comparten escala. Versión interactiva de los diez vídeos en el Anexo A: centros y evolución.

La figura compara la posición de las anotaciones en el exterior (V04) y el interior (V06) con muchos negativos. Se normalizan los centros por tamaño de imagen y el histograma 16 × 9 por número de imágenes. Aunque la mayoría de cajas están cerca del centro, la proporción de cajas cercanas a los

bordes es mayor en las difficult, porque muchas son robots cortados por el borde o que entran y salen de la imagen. Los robots aparecen siempre en una franja a media altura y de forma simétrica: el volteo horizontal es válido y el desplazamiento vertical del aumento debe ser pequeño (Tabla 8).

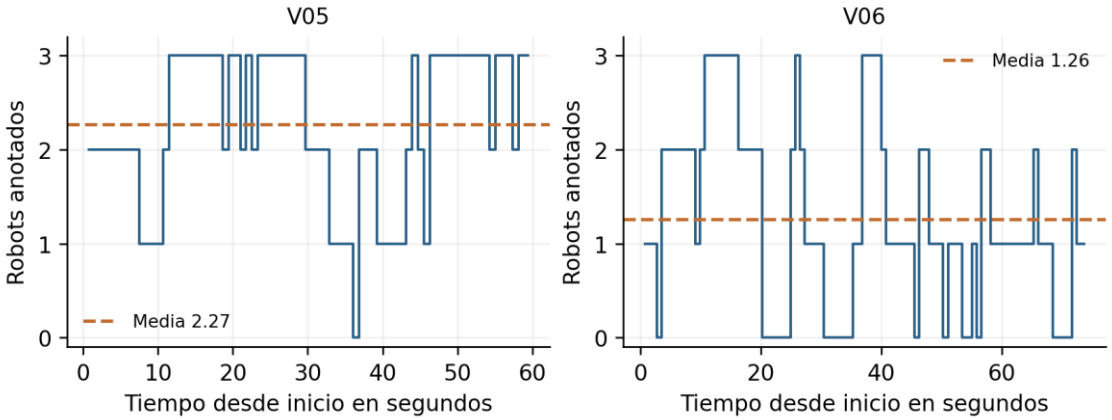


Figura 7. Robots por fotograma anotado en V05 y V06, cada 24 fotogramas, con la media de cada vídeo. Versión interactiva en el Anexo A.

Los recuentos muestran qué tramos de cada vídeo tienen más y menos robots y también permiten encontrar discrepancias en las anotaciones cuando los robots se mueven. En torno al segundo 20 de V05, por ejemplo, hay tres robots en pantalla, pero en algunos fotogramas no se anotaron, probablemente porque estaban borrosos o tapados. Para el entrenamiento, un robot visible sin caja enseña al detector que es fondo; por eso el criterio marca como difficult los robots movidos en lugar de dejarlos sin anotar (Tabla 4).

4.4. Tamaño de las cajas frente a su posición

Los mapas anteriores solo usan el centro de cada caja, pero el tamaño también depende de la posición. La cámara va a ras de suelo, así que un robot más cercano aparece más abajo y más grande (Figura 8, izquierda). La correlación entre la base de la caja y el logaritmo de su alto es de 0,68 (0,66 con el ancho): con la base por encima de la fila 360, la caja mediana mide 79 × 71 px; entre las filas 360 y 480, 121 × 107 px; y por debajo, 283 × 252 px. Las 15 cajas que más se apartan de esta relación se revisaron a ojo y son correctas: corresponden a momentos en que la cámara se inclina.

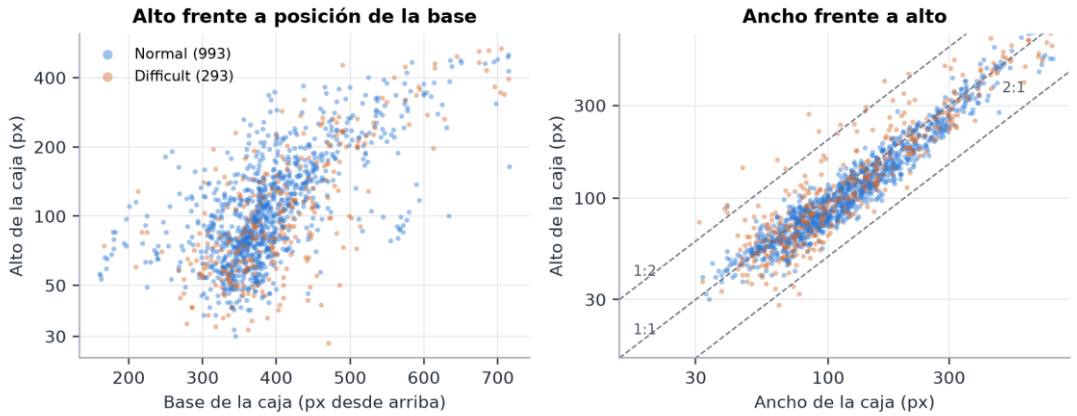


Figura 8. Izquierda: alto de la caja frente a la posición de su base. Derecha: ancho frente a alto de las 1420 cajas.

El ancho y el alto varían juntos (correlación de 0,94 entre sus logaritmos, Figura 8, derecha): las cajas son casi cuadradas en todo el rango de tamaños y quedan entre las proporciones 1:2 y 2:1 de las anclas

del detector. Las difficult tienen un tamaño mediano parecido (105 × 107 px frente a 114 × 98 px), pero una forma más variable: el 10 % más estrecho baja de 0,69 de ancho entre alto, frente a 0,91 en las normales, porque son robots cortados o movidos. De las 134 cajas que tocan el borde, el 61 % son difficult, frente al 23 % del resto.

Para el modelo, posición y tamaño no son independientes: subir o bajar un robot sin cambiar su tamaño crearía escenas imposibles, por eso el desplazamiento vertical se limita al ±5 % y la escala no deforma (Tabla 8). Las anclas por defecto ya cubren las formas observadas y no hace falta cambiarlas. Además, se midió el IoU entre robots de una misma imagen, porque el detector elimina con supresión de no máximos (NMS) las cajas que se solapan más de 0,5 con otra de mayor confianza. De 809 parejas, solo 92 se tocan y el IoU máximo es 0,26: la NMS no puede borrar robots reales que estén juntos.

4.5. Nuestros vídeos frente a validación y test

Validación y test son secuencias de otros grupos (apartado 7), grabadas con otras cámaras y en otros escenarios. Para saber en qué se parecen a nuestros datos se calcularon las mismas medidas en los tres conjuntos (Tabla 6; las distribuciones completas están en la Figura 19, Anexo C). Las de test se calcularon después de la evaluación final y no se usaron para tomar ninguna decisión.

Medida	Nuestros vídeos	Validación	Test
Secuencias / imágenes / robots	10 / 808 / 1420	11 / 672 / 1561	13 / 718 / 1721
Robots por imagen	1,76	2,32	2,40
Robots difficult	26,4 %	16,3 %	15,7 %
Cajas de menos de 30 / 50 px de ancho	0,0 / 3,9 %	2,5 / 11,3 %	0,5 / 9,1 %
Centro vertical, percentil 90	0,58	0,61	0,71
Brillo mediano (0-255)	104,8	97,8	103,9
Imágenes con $\lvert B - R \rvert > 12$	26,4 %	19,9 %	1,1 %

Tabla 6. Medidas de nuestros vídeos, validación y test con las mismas etiquetas y el mismo cálculo. B − R es la media del canal azul menos la del rojo de cada imagen.

Tres diferencias afectan al entrenamiento. La primera son los robots lejanos: en validación, el 11,3 % de las cajas mide menos de 50 px de ancho y el 2,5 %, menos de 30 px, frente al 3,9 % y ninguna en nuestros vídeos. Por eso el aumento aleja la imagen hasta ×0,6, y era previsible que fueran un punto débil: en el Bloque 2, el AP de objetos pequeños se queda entre 0,12 y 0,19 (apartado 11.2). La segunda es el color: una secuencia de validación tiene un tono rojizo (B − R = −28,0) que no aparece en nuestros vídeos, entre −4,0 y 8,3 en interior. Es un balance de blancos distinto, lo que simula el desplazamiento RGB (apartado 5.2). La tercera es que test tiene más robots en la parte baja de la imagen (percentil 90 del centro en 0,71 frente a 0,58), más cercanos y grandes, que la escala hasta ×1,25 cubre en parte. El brillo, en cambio, es parecido.

4.6. Consecuencias para el entrenamiento

La Tabla 7 resume qué decisión se tomó a partir de cada análisis y qué se observó después en el Bloque 2. El EDA anticipó los dos puntos débiles del detector, los difficult y los robots lejanos, que siguen siendo los errores principales con cualquier arquitectura (apartado 11.3).

Resultado del EDA	Decisión	Qué se observó en el Bloque 2
Imagen más parecida del mismo vídeo en el 74 % de los casos (4.2).	Validar y probar con secuencias de otros grupos.	AP@0,5 de 0,9513 en validación y 0,9523 en test.
26,4 % de difficult, junto a los bordes y en algunos vídeos (4.1, 4.4).	Conservarlos y medir su AP aparte.	Error principal: 74 de los 115 robots perdidos en test; AP@0,5 de 0,69 frente a 0,97.
Ninguna caja de menos de 30 px; en validación, un 2,5 % (4.5).	Alejar hasta ×0,6; probar 1024 px.	AP de objetos pequeños de 0,12 a 0,19; SSDlite320 (320 px) baja a 0,68.
Cajas casi cuadradas; anclas con IoU ≥ 0,5 para el 99,6 % (4.2, 4.4).	Anclas por defecto; escala sin deformar.	Los cuatro modelos con ResNet-50 superan 0,93 de AP@0,5.
IoU máximo de 0,26 entre robots (4.4).	NMS con umbral 0,5.	No borra robots reales; las cajas duplicadas (12) invitan a probar 0,3.

Tabla 7. Consecuencias del EDA para el entrenamiento y lo que se observó después en el Bloque 2.

5. Aumento de datos

El aumento se aplica solo al conjunto de entrenamiento y siempre a partir de los originales, nunca sobre copias ya aumentadas. Cada una de las 796 imágenes depuradas recibe tres copias, lo que da 3184 imágenes y 5566 cajas. Validación (672 imágenes) y test (718) son secuencias de otros grupos y se copian sin transformar, porque deben parecerse a lo que verá el robot y no a nuestras transformaciones.

El criterio para elegir cada transformación es doble, debe simular una variación que el robot puede encontrar en la demo y que nuestros diez vídeos no cubren del todo, y no debe producir escenas imposibles ni cajas que dejen de ajustarse al robot. Cada copia recibe una combinación distinta, con una media de 2,7 transformaciones, y la intensidad de cada una se sortea dentro de su rango.

5.1. Transformaciones geométricas

Transformación	Rango	p	Intención
Volteo horizontal	—	0,5	Un robot visto en espejo sigue siendo un robot válido y el EDA muestra que los centros de las cajas son simétricos en horizontal. Duplica las orientaciones sin crear escenas irreales.
Escala sin deformar	×0,6–1,25	0,7 (conjunta)	Alejar produce robots más pequeños y simula mayor distancia: no había cajas originales de menos de 30 px de ancho y la demo puede exigir detectar más lejos. Acercar produce robots cortados por el borde, frecuentes en nuestros vídeos.
Desplazamiento	±8 % horizontal, ±5 % vertical	0,7 (conjunta)	Cambia la posición del robot sin sacarlo de la franja a media altura en la que aparece siempre.
Giro	±6°	0,7 (conjunta)	La cámara se inclina al acelerar y en V06; el EDA atribuye a esa inclinación las cajas atípicas. Con giros mayores la caja deja de ajustarse al robot.

Tabla 8. Transformaciones geométricas utilizadas y su intención. Escala, desplazamiento y giro forman una sola transformación afín, que se aplica con p = 0,7 y sortea los tres valores a la vez.

Se aplican dos decisiones de implementación para evitar errores silenciosos. El hueco que deja un giro o un alejamiento se rellena de gris y no con un reflejo de la imagen, que podría duplicar un robot en el borde sin su caja y enseñar a la red a tratarlo como fondo. Al girar, la caja se recalcula aproximando el robot por una elipse inscrita en ella, lo que la mantiene ajustada; el método por esquinas la agranda (IoU de 0,85 con ±6°).

5.2. Color e iluminación

Transformación	Rango	p	Intención
Brillo y contraste	±20 %	0,25	Cubre iluminaciones no grabadas: otra hora del día, luces apagadas o contraluz.
Gamma	75–125	0,25	Complementa al brillo: aclara u oscurece las sombras sin saturar las zonas iluminadas.
Tono y saturación	±8 de tono, ±20 de saturación, ±10 de valor	0,15	Cambios suaves, como recomienda el EDA por el tono azulado de los exteriores. Un cambio fuerte alteraría el color de los LED, que es la pista más clara para reconocer el robot.
Desplazamiento RGB	±12 por canal	0,15	Simula el balance de blancos de otra cámara: una secuencia de validación tiene un tono rojizo (B − R = −28) que no hay en nuestros vídeos (apartado 4.5).

Tabla 9. Transformaciones de color e iluminación utilizadas y su intención. Se aplica una de las dos primeras con p = 0,5 y una de las dos últimas con p = 0,3.

El desplazamiento RGB suma una constante a cada canal, como un balance de blancos distinto, sin depender del contenido de la escena. Se mantiene del aumentador de la asignatura, pero con menos amplitud (±12 frente a ±15) y como alternativa al cambio de tono, para no acumular dos cambios de color en la misma copia ni alterar el color de los LED. La comparación con validación confirma que hace falta: con R +12 y B −12, una imagen de nuestros interiores pasa de B − R ≈ 0 a −24, cerca de los −28 de la secuencia rojiza (Figura 19, Anexo C).

5.3. Calidad de imagen

De estas cuatro transformaciones se aplica como mucho una por copia (p = 0,3), para no acumular degradaciones que harían la imagen peor que cualquier fotograma real. El desenfoque de movimiento se sortea aparte. Si a una copia no le toca ninguna transformación, se repite el sorteo, de modo que la probabilidad real de cada una es algo mayor que la indicada.

Transformación	Rango	p	Intención
Desenfoque de movimiento	Núcleo de 3–9 px	0,15	Es el defecto real de nuestros vídeos, porque se mueven la cámara y los robots. Sustituye al desenfoque uniforme del aumentador de la asignatura.
Compresión JPEG	Calidad 35–75	0,075	El vídeo llega comprimido desde el robot y pierde calidad cuando la señal es mala.

Transformación	Rango	p	Intención
Ruido de sensor	Gaussiano (varianza 10–40) o ISO	0,15 (0,075 cada tipo)	Reproduce el ruido que aparece con poca luz.
Menor resolución	×0,5–0,75	0,075	El SDK puede entregar el vídeo a menos resolución que la aplicación del robot.

Tabla 10. Transformaciones de calidad de imagen utilizadas y su intención. Compresión, ruido y menor resolución se sortean juntas: se aplica una de las cuatro con p = 0,3.

5.4. Transformación propia: oclusión sintética

En la demo los robots estarán detrás de obstáculos fijos, y en nuestros vídeos ya aparecen detrás de patas de silla y de piernas. Para entrenar con más casos parcialmente tapados se dibujan una o dos barras oscuras casi verticales en el 15 % de las copias. Ninguna barra puede tapar más del 35 % del ancho de un robot, de modo que sigue siendo reconocible y su caja no cambia, como establecen los criterios de anotación para obstáculos finos (Tabla 4).

Se prefiere a borrar rectángulos al azar (CoarseDropout en Albumentations [2]), que puede tapar un robot entero dejando su caja sobre el fondo. Es la transformación menos realista del conjunto, ya que una barra lisa no es una pata de silla; por eso el entrenamiento usa postes con textura (apartado 6.1).

5.5. Ejemplo y transformaciones descartadas

La Figura 20 (Anexo C) aplica cada transformación por separado a una imagen de V06, con una intensidad fija para que se aprecie. En el aumento real se combinan y su intensidad se sortea dentro del rango de las tablas.

Se descartaron ocho transformaciones (Anexo B). Las principales son el volteo vertical y los giros de 90°, porque el robot nunca aparece boca abajo ni de lado; la perspectiva, porque en Albumentations 1.4.18 devuelve cajas que no coinciden con el robot; y el recorte 4:3 del aumentador de la asignatura, que ensancha los robots un 33 % y pierde el 14,1 % de las cajas.

La ejecución completa del aumento de datos está disponible en [8].

6. Depuración de datos

Se revisaron visualmente 69 imágenes para comprobar su calidad y utilidad. De ellas, 52 presentaban poca nitidez, 11 eran casi duplicadas y 6 planteaban dudas. Al final, descartamos una imagen en la que aparecía un robot que no podía anotarse de forma consistente y se eliminaron otras 11 por ser redundantes. En cambio, se mantuvieron 41 imágenes movidas con sus robots marcados como *difficult*, así como los negativos difíciles. Esta decisión se tomó porque una imagen poco nítida no tiene por qué ser inútil para el entrenamiento, pues también permite representar las condiciones reales en las que trabajará el detector.

Conjunto	Imágenes	Cajas
Original anotado	808	1420
Original tras depuración	796	1399

Conjunto	Imágenes	Cajas
Copias aumentadas	2388	4167
Entrenamiento total	3184	5566

Tabla 11. Efecto de la depuración y del aumento.

6.1. Ampliaciones del aumento

Al aumento base se añadieron sombras y llovizna en los vídeos exteriores (V04 y V09), postes con textura en lugar de las barras lisas y una copia más oscura o más movida de las imágenes con difficult. Con este perfil se entrenaron todos los modelos del apartado 11, incluido el final. En Faster R-CNN v1 el AP@0,5 en validación apenas cambió respecto al aumento base (0,9426 frente a 0,9433), también en difficult (0,6471 frente a 0,6462). Una posible explicación es que el aumento base ya cubría parte de esa variabilidad, pero no se ha comprobado por falta de explicabilidad su causa raíz.

7. El pipeline de entrenamiento

El entrenamiento se realiza con PyTorch [18] y torchvision [4] utilizando 3184 imágenes (las 796 originales y tres versiones aumentadas de cada una). La validación incluye 672 imágenes de 11 secuencias grabadas, con 1561 robots y 255 difficult. Por su parte, el test contiene 718 imágenes de 13 secuencias externas, con 1721 robots y 270 difficult. Este conjunto se reserva para evaluar cada modelo una sola vez, sin utilizar sus resultados para tomar decisiones (apartado 11.6).

Los difficult de validación y test los marcamos nosotros, porque venían todos sin marcar. Con los criterios de nuestras etiquetas (Tabla 4), se marcaron directamente los 48 robots de menos de 30 píxeles de ancho y se revisaron a ojo, recortados y ampliados, los 930 que estaban cortados por el borde, tapados al menos un 30 % por otro robot o poco nítidos; de ellos se marcaron 477. Las cajas no se tocaron. Así se puede medir el detector por separado en los casos difíciles, que es donde más falla (apartado 11).

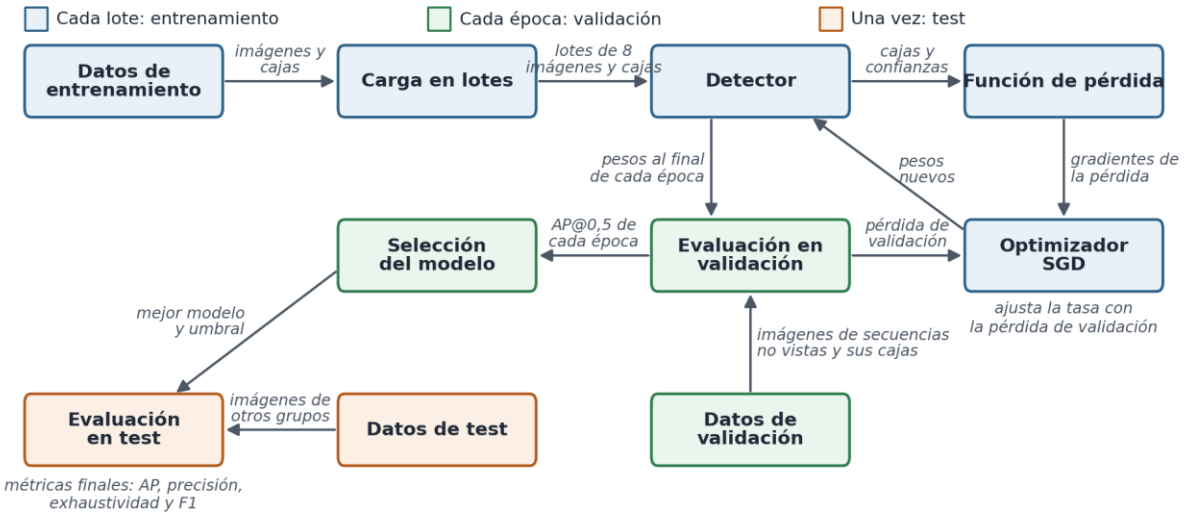


Figura 9. Esquema de bloques del entrenamiento y de la evaluación: los módulos y los datos que pasa cada uno al siguiente.

La Figura 9 resume el sistema: cada bloque es un módulo y cada flecha, los datos que pasan al siguiente.

Datos de entrenamiento. Reúne las imágenes originales y aumentadas con sus anotaciones y entrega cada imagen con la lista de sus robots: la posición de cada caja y su clase.

Carga en lotes. Convierte cada imagen en una matriz de valores entre 0 y 1, baraja el conjunto en cada época y agrupa las imágenes en lotes de 8 con sus cajas.

Detector. Normaliza y reescala cada imagen (apartado 9), extrae sus características y devuelve cajas candidatas con su confianza de ser un robot (apartado 8).

Función de pérdida. Compara esas cajas con las anotadas y mide tres errores: el de las regiones propuestas, el de la clase (robot o fondo) y el de la posición de cada caja. Su suma es la pérdida, y la retropropagación calcula cuánto contribuye cada peso a ella: el gradiente.

Optimizador SGD [22]. Mueve cada peso en contra de su gradiente, con un paso proporcional a la tasa de aprendizaje, y devuelve los pesos nuevos al detector. Su programador baja la tasa cuando la pérdida de validación se estanca.

Evaluación en validación. Al final de cada época aplica el detector, sin modificar sus pesos, a las imágenes de validación, de secuencias que no ha visto, y devuelve la pérdida de validación y el AP@0,5.

Selección del modelo. Conserva los pesos de la época con mayor AP@0,5 y fija el umbral de confianza con el mejor F1 en validación (apartado 11.6).

Evaluación en test. Aplica una sola vez el modelo elegido, con su umbral, a las imágenes de test y da las métricas finales (apartado 10).

Un batch es el grupo de imágenes procesadas en cada paso, mientras que una época corresponde a una pasada completa por el conjunto de entrenamiento. Se trabaja por batches porque las 3184 imágenes no caben a la vez en la memoria de la GPU y porque así se actualizan los pesos varias veces por época.

Las cajas difficult también participan en el entrenamiento: antes de entregarlas al detector solo se quita su marca de difficult, no las cajas.

8. Modelos y transfer learning

Un detector está formado por un backbone, la red convolucional (ResNet-50 [11], MobileNetV3 [13] o VGG16 [14]) que extrae características de la imagen, y por las partes que localizan y clasifican los objetos. Se distingue el backbone porque se puede intercambiar entre modelos, influye mucho en el coste computacional y cuenta con pesos preentrenados.

Casi todos añaden un cuello FPN [12] entre el backbone y esas partes. El backbone produce mapas de características cada vez más pequeños: los primeros conservan el detalle espacial y los últimos, más información sobre qué hay en la imagen. La FPN los recorre de arriba abajo: amplía cada mapa profundo al tamaño del nivel anterior y lo suma a ese nivel, adaptado con una convolución de 1 × 1. El resultado es una pirámide de mapas que tienen todos información sobre qué hay, cada uno con su resolución, y cada robot se busca en el nivel adecuado a su tamaño: los lejanos en los mapas grandes y los cercanos en los pequeños.

Los modelos son de dos etapas o de una (Tabla 12). Los de dos etapas, como Faster R-CNN [3], primero generan con una red de propuestas (RPN) unas mil regiones que pueden contener un objeto, y después recortan las características de cada región y las clasifican y ajustan una a una. Los de una etapa, como RetinaNet [9] y SSD [10], no usan propuestas: en cada posición de los mapas de características colocan

cajas de referencia (anclas) de varios tamaños y predicen de una sola vez la clase y el ajuste de todas. Los de dos etapas suelen localizar mejor, porque la segunda etapa trabaja con pocas regiones ya filtradas; los de una etapa son más sencillos y, en general, más rápidos, pero deben distinguir unos pocos objetos entre decenas de miles de anclas de fondo, el desequilibrio que compensa la pérdida focal de RetinaNet. En nuestras medidas, con el mismo backbone, la diferencia de tiempo es pequeña (Tabla 16): pesa más el tamaño del backbone y de la imagen.

Todos los modelos utilizan transfer learning, es decir, parten de pesos aprendidos con COCO [15], una base de datos general con más de cien mil imágenes anotadas, y sustituyen la cabeza de clasificación por otra con dos clases: fondo y robot. De esta forma, la red aprovecha lo aprendido sobre bordes, texturas y formas, y adapta sus predicciones al objeto que queremos detectar [3,4]. Esto resulta especialmente importante porque solo disponemos de 796 imágenes originales de diez vídeos, una cantidad insuficiente para entrenar desde cero redes con decenas de millones de parámetros. Además, permite acelerar la convergencia del modelo (apartado 11.3).

Modelo	Tipo	Cómo funciona	Backbone y cuello
Faster R-CNN v1	Dos etapas	Primero propone regiones con posibles objetos (RPN) y después las clasifica y ajusta sus cajas.	ResNet-50 + FPN
Faster R-CNN v2	Dos etapas	Sigue el mismo proceso, pero incorpora mejoras en las capas de predicción y en el entrenamiento [16].	ResNet-50 + FPN
Faster R-CNN MobileNet	Dos etapas, versión ligera	Utiliza la misma idea, pero con una red más ligera para reducir el coste computacional y priorizar la velocidad.	MobileNetV3-L + FPN
RetinaNet v1 / v2	Una etapa	Detecta y clasifica objetos directamente en distintas posiciones y escalas. Utiliza la pérdida focal para dar más importancia a los ejemplos difíciles frente al gran número de regiones de fondo [9]. La v2 incorpora mejoras en la cabeza y el entrenamiento.	ResNet-50 + FPN
SSD300	Una etapa	Detecta objetos de distintos tamaños en una imagen de entrada de 300 × 300 píxeles [10].	VGG16
SSDlite320	Una etapa, versión ligera	Utiliza convoluciones más eficientes para reducir el coste computacional y procesa imágenes de 320 × 320 píxeles [17].	MobileNetV3-L

Tabla 12. Los siete modelos comparados.

9. Hiperparámetros

Parámetro	Valor aplicado
Épocas y tamaño (lote)	12 épocas (30 al inicio; 15 con lr $5{\cdot}10^{-4}$); lote efectivo de 8 imágenes. Faster R-CNN v2 y el experimento a 1024 píxeles, microlotes de 4 × 2.
Semilla aleatoria	0
Optimizador	SGD [22] con momentum de 0,9 [23] y weight decay de $5{\cdot}10^{-4}$.
Tasa de aprendizaje	10^{-4} en los dos primeros experimentos; 10^{-3} en el resto y en las arquitecturas; $5{\cdot}10^{-4}$ y $2{\cdot}10^{-3}$ en variantes.

Ajuste de la tasa	Reduce la tasa ×0,1 tras 5 épocas sin mejorar la pérdida de validación; mínimo 10^{-10}.
Resolución (entrada)	Imágenes de 1280 × 720. Faster R-CNN y RetinaNet las reescalan a 1333 × 750 (lado corto a 800 salvo que el largo pase de 1333) y a 1820 × 1024 en el experimento de resolución. SSD300 y SSDlite320, a 300 × 300 y 320 × 320.
Datos de entrenamiento	Imágenes originales y tres copias aumentadas de cada una, con el mismo aumento (apartado 6.1).
Selección del modelo	Se conserva el checkpoint con mayor AP@0,5 en validación.
Entorno de ejecución	PyTorch 2.4.1, torchvision 0.19.1 y GPU NVIDIA RTX 3060 de 12 GB.

Tabla 13. Configuración común de los experimentos medidos.

La Tabla 13 recoge los hiperparámetros comunes utilizados en los experimentos.

Batch size (tamaño del lote). El gradiente de cada paso se calcula con las imágenes del lote y no con todo el conjunto, así que es una estimación del gradiente verdadero. Como es la media de los gradientes de cada imagen, su dispersión baja con la raíz cuadrada del tamaño del lote: con una imagen, cada paso sigue a esa imagen y la dirección cambia mucho de un paso a otro; con 32, las particularidades de cada imagen se compensan. Ese ruido tiene dos caras. Con moderación es bueno: actúa como regularización y ayuda a salir de mínimos estrechos, que generalizan peor [24], y un lote pequeño da además más actualizaciones por época. En exceso es malo, porque obliga a usar tasas bajas para que el entrenamiento no se vuelva inestable. Un lote grande da gradientes más fiables y aprovecha mejor la GPU, pero necesita más memoria, da menos pasos por época y suele requerir subir la tasa en proporción [25].

En las pruebas cortas (Tabla 22, Anexo D), con lotes de 1 y 2 el AP@0,5 sube antes, porque hay más pasos por época, pero baja en la tercera época; con 8 sube de forma más regular, y los tres terminan a menos de 0,4 pp. Los lotes de 16 y 32 no caben en los 12 GB de la GPU. Se elige 8 porque es el mayor que cabe: da gradientes estables con la tasa de 10^{-3} y unos 400 pasos por época con las 3184 imágenes. Faster R-CNN v2 y el experimento a 1024 píxeles no caben con 8, así que usan microlotes de 4 y suman los gradientes de dos microlotes antes de cada paso: el lote efectivo sigue siendo 8. Se prueban, por orden, microlotes de 8, 4 y 2, que son los que dividen a 8, y se usa el primero que cabe. Con 6, los pasos tendrían 6 o 12 imágenes y no serían comparables con los del resto de modelos.

Learning rate (tasa de aprendizaje). Determina el tamaño de los pasos que se dan al actualizar los pesos. Una tasa demasiado alta hace que los pasos se pasen del mínimo: el entrenamiento se vuelve inestable y puede divergir, como con 10^{-1}, cuya pérdida acaba en NaN (Tabla 23). Una tasa demasiado baja da pasos tan cortos que el modelo apenas avanza: necesita muchas más épocas para llegar al mismo resultado, puede quedarse estancado en zonas planas de la pérdida y, con un número de épocas fijo, acaba sin haber aprendido lo suficiente (infraajuste). Con 10^{-5}, tras tres épocas el AP@0,5 no llega a 0,03 (Tabla 23), y con 10^{-4} el mejor checkpoint llega en la época 29, frente a la 5 con 10^{-3}. Con $2{\cdot}10^{-3}$ el máximo llega en la época 1 o 2 y después el AP@0,5 baja más de 1 pp. Entre 10^{-4} y 10^{-2} la tasa cambia sobre todo la velocidad del entrenamiento, no su resultado (apartado 11.4). El

programador reduce la tasa cuando la pérdida de validación deja de mejorar, aunque no siempre lo hace a tiempo para recuperar el mejor resultado (apartado 11.1).

Épocas. Una época corresponde a una pasada completa por las imágenes de entrenamiento. Los primeros experimentos usaron 30 épocas y los experimentos de comparación entre modelos, 12 (15 con lr $5 \cdot 10^{-4}$). En seis de los siete modelos, el mejor checkpoint está entre las épocas 3 y 7; SSD300, en la 12 (en la 18 con 40 épocas, con el mismo AP@0,5; apartado 11.5). La pérdida de validación toca su mínimo pronto (época 4 con 30 épocas) y después aumenta mientras la de entrenamiento baja: hay sobreajuste, no falta de épocas. Por eso, se selecciona el mejor checkpoint de validación en lugar de asumir que el último es el mejor.

Optimizador y programador. Se utiliza SGD [22], que actualiza los pesos con el gradiente de cada lote. El momentum (0,9) [23] acumula una media de los gradientes anteriores, de modo que cada paso sigue la dirección de unos diez lotes, $1/(1 - 0{,}9)$, y no solo la del último: suaviza el ruido de los lotes pequeños y acelera el avance en las direcciones que se repiten. El weight decay ($5 \cdot 10^{-4}$) resta en cada paso una fracción de cada peso; así penaliza los pesos grandes y limita el sobreajuste. Son los valores del artículo de Faster R-CNN [3] y del tutorial de torchvision para ajustar detectores [27], y no se cambiaron: por coste, solo se ajustó la tasa, el hiperparámetro al que el entrenamiento es más sensible. El programador multiplica la tasa por 0,1 cuando la pérdida de validación lleva 5 épocas sin mejorar. Ese margen evita reaccionar al ruido de una sola época, pero resultó largo: el mejor checkpoint llega entre las épocas 3 y 7 y la primera bajada, en la 11 (apartado 11.1).

Una estrategia más sistemática para elegir estos valores sería: primero, una prueba de rango de la tasa [26], que entrena unos cientos de pasos subiendo la tasa de forma exponencial y elige un valor un orden de magnitud por debajo del que dispara la pérdida; después, una búsqueda corta en escala logarítmica alrededor de ese valor, comparando en validación, como la del Anexo D; y, por último, ajustar la paciencia del programador a la época en que se estanca la validación, aquí 2 o 3 épocas. El momentum y el weight decay se ajustarían al final, porque sus valores habituales suelen funcionar en el ajuste fino de detectores. No se probaron otros optimizadores, por lo que no se puede afirmar que SGD sea superior a las alternativas.

10. Métricas

El IoU mide cuánto se solapan la caja predicha y la caja real, dividiendo su intersección entre su unión. Para evaluar las detecciones, se exige un IoU ≥ 0,5 y una confianza ≥ 0,5. Las detecciones se emparejan por confianza, es decir, si una coincide con un robot anotado, es un verdadero positivo (TP); si no tiene pareja, es un falso positivo (FP); y si un robot anotado queda sin detectar, es un falso negativo (FN).

La precisión mide qué proporción de las detecciones son correctas (TP/(TP+FP)), mientras que la recall indica qué proporción de los robots anotados se detecta (TP/(TP+FN)). El F1 combina ambas mediante su media armónica. El umbral de confianza establece la confianza mínima para aceptar una detección, al subirlo, suelen disminuir los FP y aumentar la precisión, pero también pueden aumentar los FN y bajar la exhaustividad. Al reducirlo, ocurre lo contrario. Este es el compromiso entre ambas métricas (apartado 11.6).

La AP (Average Precision) resume la curva precisión-exhaustividad al variar el umbral de confianza y se calcula como el área bajo esa curva. AP@0,5 utiliza un IoU de 0,5, mientras que AP@0,75 exige una mayor precisión. AP 0,50:0,95 promedia los resultados para umbrales de IoU entre 0,50 y 0,95; es la métrica conocida como mAP en COCO. La «m» indica que se promedia entre clases y, al trabajar con

una sola clase, la mAP coincide con la AP. Las diferencias entre dos valores de AP, precisión o F1 se dan en puntos porcentuales (pp): de 0,9433 a 0,9465 hay 0,32 pp.

La Figura 10 muestra la curva del modelo final y la de Faster R-CNN MobileNet en test. Cada punto es un umbral de confianza: al bajarlo, el detector acepta más cajas, encuentra más robots (más exhaustividad) y comete más falsos positivos (menos precisión). Un detector ideal estaría en la esquina superior derecha. La curva del modelo final queda por encima y a la derecha de la de MobileNet en todo el recorrido; por eso su AP es mayor.

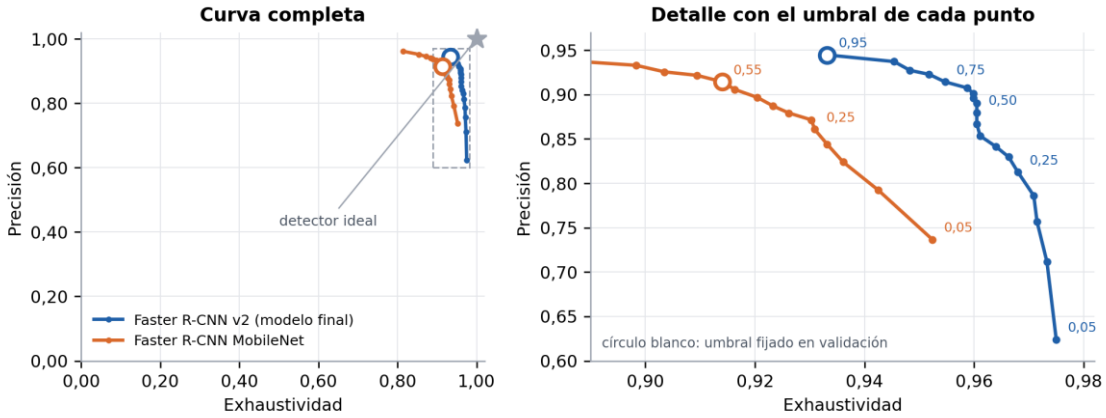


Figura 10. Curva precisión-exhaustividad en test (IoU ≥ 0,5), con umbrales de confianza de 0,05 a 0,95. El círculo blanco marca el umbral fijado en validación.

Con el modelo final, Faster R-CNN v2, en test (718 imágenes, 1721 robots, 270 de ellos difficult) y el umbral de confianza de 0,95 fijado en validación hay 1606 TP, 94 FP y 115 FN (precisión de 0,9447, exhaustividad de 0,9332 y F1 de 0,9389), con una AP@0,5 de 0,9523, una AP@0,75 de 0,6005 y una AP 0,50:0,95 de 0,5724. Casi todos los robots se encuentran, pero muchas cajas tienen un IoU menor de 0,75. Más desgloses, en el Anexo E.

11. Experimentos

Los experimentos se organizaron en tres fases. Primero, con un único modelo, Faster R-CNN v1 (el de partida que propone el enunciado), se fijaron los hiperparámetros para no multiplicar el coste: con y sin aumento de datos, tasa baja (10^{-4}) o alta (10^{-3}) e imagen normal o grande (apartados 11.1 y 11.2). Después, con esos hiperparámetros, se compararon las siete arquitecturas (11.3) y se probaron tres tasas con las tres mejores (11.4). Por último, se comprobó con SSD300 si faltaban épocas (11.5) y se evaluó cada modelo en test una sola vez (11.6).

11.1. Aumento de datos y tasa de aprendizaje Faster R-CNN v1

Experimento	Mejor época	AP@0,5 al entrenar	AP@0,5 al evaluar	AP 0,50:0,95
Sin aumento, tasa baja (10^{-4})	20	0,9229	0,9228	0,5809
Con aumento, tasa baja (10^{-4})	29	0,9425	0,9416	0,6081

Experimento	Mejor época	AP@0,5 al entrenar	AP@0,5 al evaluar	AP 0,50:0,95
Con aumento, tasa alta (10^{-3})	5	0,9432	0,9433	0,6302
Con aumento, tasa alta, imagen grande	2	0,9464	0,9465	0,6047

Tabla 14. Experimentos de ajuste con Faster R-CNN v1 en validación, con el mejor checkpoint por AP@0,5. «Al entrenar» es el AP que se calcula al final de cada época y «al evaluar», el de la evaluación posterior de ese checkpoint.

El aumento de datos es lo que más mejora a Faster R-CNN v1, sobre todo en los robots difficult (Tabla 14), aunque parte de la mejora puede deberse a que el aumento cuadruplica las imágenes y las actualizaciones. Con la tasa alta, el AP@0,5 es el mismo, las cajas se ajustan algo mejor (AP 0,50:0,95) y el mejor checkpoint llega seis veces antes, en la época 5 en vez de la 29.

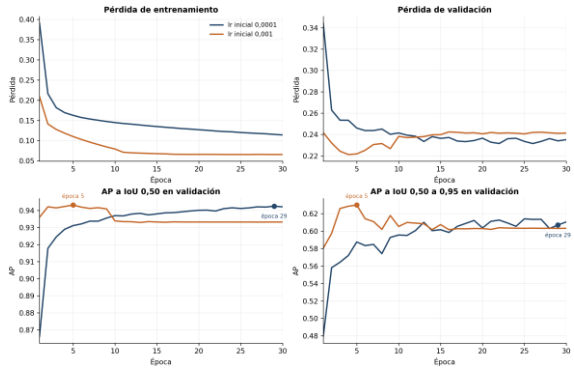


Figura 11. Pérdidas y AP por época. Los puntos señalan los checkpoints elegidos por AP@0,5. Interactiva en el Anexo A.

Con la tasa alta, la pérdida de validación toca su mínimo en la época 4 y después sube, mientras la de entrenamiento sigue bajando (Figura 11): el modelo se ajusta cada vez más a las imágenes de entrenamiento y empeora en las que no ha visto, es decir, sobreajusta. Por eso se conserva el mejor checkpoint y no el último. El programador baja la tasa por primera vez en la época 11 (Figura 21, Anexo C), demasiado tarde para recuperar el mejor resultado.

11.2. Resolución de entrada Faster R-CNN v1

Los vídeos se graban a 1280 × 720 píxeles, igual que los de validación y test. Antes de analizar cada imagen, Faster R-CNN y RetinaNet la reescalan sin deformarla con dos límites: el lado corto pasa a 800 píxeles, salvo que entonces el largo supere los 1333. En una imagen de 16:9 manda el segundo límite y la imagen entra a 1333 × 750, apenas un 4 % más grande que la original. Como los robots lejanos seguían detectándose mal, se subieron los límites a 1024 y 1820, sin cambiar nada más: la imagen entra entonces a 1820 × 1024, un 86 % más de píxeles. Por memoria, este experimento usó microlotes de 4 imágenes con dos acumulaciones de gradientes.

Mejor por AP@0,5 en validación	800 px	1024 px	Cambio
AP@0,5 todos	0,9433	0,9465	+0,32 pp
AP 0,50:0,95 todos	0,6302	0,6047	−2,55 pp
AP@0,75 todos	0,7460	0,6990	−4,70 pp
AP@0,5 difficult	0,6462	0,6282	−1,80 pp
AP small	0,1182	0,1886	+7,04 pp

Mejor por AP@0,5 en validación	800 px	1024 px	Cambio
FP / FN (conf. 0,5)	190 / 83	324 / 68	+134 / −15
F1 (conf. 0,5)	0,9155	0,8840	−3,15 pp
Latencia por imagen	75,1 ms	121,6 ms	+61,9 %

Tabla 15. Resolución 800 frente a 1024 con el mismo procedimiento.

Con la imagen grande se detectan más robots pequeños y difficult, pero aparecen muchos más falsos positivos, las cajas se ajustan peor, baja el F1 y cada imagen tarda un 62 % más (Tabla 15).

Con el criterio fijado de antemano, el AP@0,5, la ganancia es demasiado pequeña para compensar esos costes, así que se mantiene la imagen normal, que además gana en 7 de las 11 secuencias de validación.

11.3. Comparación de arquitecturas

Se entrenaron los siete modelos del apartado 8 con pesos preentrenados de COCO y la cabeza de predicción adaptada a la detección de robots. Para cada modelo se seleccionó el checkpoint con mayor AP@0,5 en validación. La Tabla 16 recoge los resultados y la Figura 12 muestra la evolución del AP por época. Las versiones interactivas de esta y de las demás gráficas del Bloque 2, en las que se puede elegir la métrica, el modelo y el conjunto, se recogen en el Anexo A.

Modelo	Época	AP@0,5 todos	AP@0,5 normales	AP@0,5 difficult	AP 0,5:0,95	F1 máx. (umbral)	ms/imagen	Min
Faster R-CNN v2	3	0,9513	0,9759	0,6422	0,6192	0,9361 (0,95)	99,5	151
RetinaNet v2	6	0,9447	0,9748	0,6213	0,6114	0,9264 (0,65)	67,3	102
Faster R-CNN v1	5	0,9426	0,9733	0,6471	0,6273	0,9305 (0,90)	75,1	106
RetinaNet v1	7	0,9396	0,9687	0,5994	0,6196	0,9229 (0,65)	72,2	98
Faster R-CNN MobileNet	3	0,9281	0,9597	0,6027	0,5789	0,9167 (0,55)	16,0	19
SSD300 VGG16	12	0,9008	0,9404	0,5199	0,5144	0,8926 (0,25)	12,5	19
SSDlite320	7	0,6823	0,7272	0,3233	0,3943	0,6839 (0,20)	4,2	6

Tabla 16. Siete arquitecturas con lr 10^{-3} en validación (672 imágenes, 1561 robots, 255 difficult). Época del mejor checkpoint, F1 máximo, ms por imagen de una en una y minutos de entrenamiento.

Faster R-CNN v2 obtiene el mejor AP@0,5 (Tabla 16), aunque los cuatro modelos con ResNet-50 quedan a menos de 1,2 pp entre sí, una diferencia que no es concluyente con una sola semilla. Faster R-CNN llega a su máximo en pocas épocas y después empeora mientras la pérdida de entrenamiento sigue bajando: sobreajusta pronto. RetinaNet empieza más despacio, pero es más estable. SSD300 todavía mejoraba en la última época (apartado 11.5).

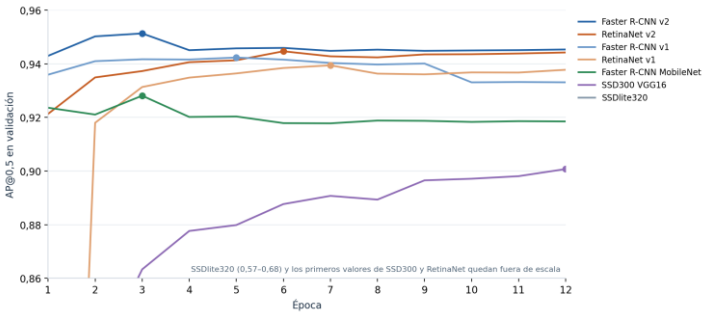


Figura 12. AP@0,5 en validación por época. El punto marca el checkpoint elegido. Interactiva en el Anexo A.

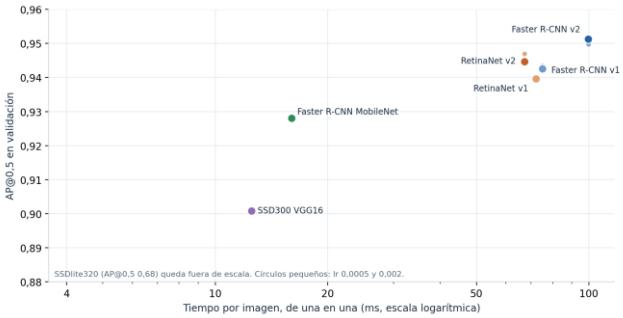


Figura 13. Precisión frente a tiempo por imagen. Interactiva en el Anexo A.

Los modelos ligeros cambian precisión por velocidad (Figura 13). Faster R-CNN MobileNet es seis veces más rápido que la v2 y pierde poco AP@0,5; SSDlite320 es el más rápido, pero con imágenes de 320 × 320 pierde los robots lejanos.

Ninguna arquitectura resuelve los difficult: en todas, su AP@0,5 queda muy por debajo del de los robots normales. Los 13 entrenamientos suman 21,8 horas de GPU.

Pruebas sin aumento (Anexo D). A diferencia de lo que propone el enunciado, todos los modelos se entrenaron con aumento, que ya mejoraba Faster R-CNN v1 (apartado 11.1). Sin él, Faster R-CNN v2 sigue siendo la mejor (Tabla 21). Con Faster R-CNN v1, los lotes de 16 y 32 no caben en la memoria de la GPU (Tabla 22) y con una tasa de 10^{-1} el entrenamiento diverge (Tabla 23).

11.4. Tasa de aprendizaje y perfil de aumento

Modelo	lr	Mejor época	AP@0,5 todos	AP@0,5 difficult	AP 0,50:0,95	Min
Faster R-CNN v2	$5{\cdot}10^{-4}$	5	0,9507	0,6361	0,6126	189
Faster R-CNN v2	10^{-3}	3	0,9513	0,6422	0,6192	151
Faster R-CNN v2	$2{\cdot}10^{-3}$	1	0,9497	0,6223	0,6403	151
RetinaNet v2	$5{\cdot}10^{-4}$	11	0,9451	0,6133	0,6170	126
RetinaNet v2	10^{-3}	6	0,9447	0,6213	0,6114	102
RetinaNet v2	$2{\cdot}10^{-3}$	6	0,9469	0,6246	0,6177	101
Faster R-CNN v1	$5{\cdot}10^{-4}$	14	0,9420	0,6323	0,6104	132

Modelo	lr	Mejor época	AP@0,5 todos	AP@0,5 difficult	AP 0,50:0,95	Min
Faster R-CNN v1	10^{-3}	5	0,9426	0,6471	0,6273	106
Faster R-CNN v1	$2{\cdot}10^{-3}$	2	0,9434	0,6465	0,6024	105

Tabla 17. Variación de la tasa de aprendizaje en los tres modelos con mejor AP@0,5.

La tasa apenas cambia el AP@0,5 de los tres modelos (Tabla 17): las diferencias, de menos de 0,22 pp, son demasiado pequeñas para distinguirlas con una sola semilla. Se mantiene 10^{-3} porque iguala a $5{\cdot}10^{-4}$ en menos épocas. Con $2{\cdot}10^{-3}$, Faster R-CNN llega a su máximo en una o dos épocas y después empeora mientras sube la pérdida de validación: inestabilidad y sobreajuste, no falta de entrenamiento.

Todos los modelos usan el perfil con sombras, llovizna y más imágenes difficult, que con Faster R-CNN v1 da el mismo AP@0,5 que el aumento base (apartado 6.1).

11.5. Comprobación con más épocas

Para comprobar si 12 épocas eran pocas, se repitió con 40 SSD300, el único modelo cuyo mejor checkpoint caía en la última época, sin cambiar nada más (Figura 14). Con la misma semilla, sus 12 primeras épocas coinciden con el entrenamiento original.

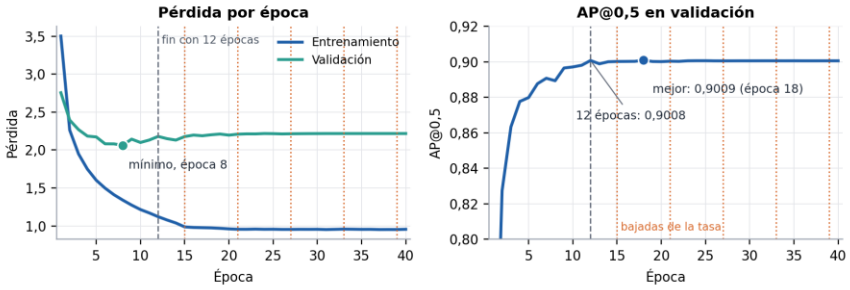


Figura 14. SSD300 con 40 épocas. La línea discontinua marca el final del entrenamiento de 12 épocas y las punteadas, las bajadas de la tasa. Interactiva en el Anexo A.

Las 28 épocas adicionales bajan la pérdida de entrenamiento, pero el AP@0,5 en validación no se mueve y la pérdida de validación sube desde la época 8, pese a las bajadas de la tasa: es el mismo sobreajuste que en Faster R-CNN (apartado 11.1), no falta de épocas. Solo mejora algo el ajuste de las cajas. Como triplica el tiempo, no se repitió con el resto de modelos, cuyo mejor checkpoint está entre las épocas 3 y 7.

11.6. Evaluación final en test

Se elige el modelo con mayor AP@0,5 en validación y el umbral con mejor F1.

Modelo	AP@0,5 val.	AP@0,5 test	Diff. test	Umbral	Precis.	Exhaust.	F1	FP / FN
Faster R-CNN v2	0,9513	0,9523	0,6921	0,95	0,9447	0,9332	0,9389	94 / 115
RetinaNet v2	0,9447	0,9457	0,6313	0,65	0,9409	0,9064	0,9233	98 / 161
Faster R-CNN v1	0,9426	0,9355	0,6057	0,90	0,9224	0,9250	0,9237	134 / 129
RetinaNet v1	0,9396	0,9362	0,6273	0,65	0,9226	0,9210	0,9218	133 / 136

Modelo	AP@0,5 val.	AP@0,5 test	Diff. test	Umbral	Precis.	Exhaust.	F1	FP / FN
Faster R-CNN MobileNet	0,9281	0,9240	0,5744	0,55	0,9145	0,9140	0,9142	147 / 148
SSD300 VGG16	0,9008	0,8797	0,4316	0,25	0,9128	0,8216	0,8648	135 / 307
SSDlite320	0,6823	0,6412	0,2742	0,20	0,7284	0,5985	0,6571	384 / 691

Tabla 18. Evaluación final en test, una vez por modelo y con el umbral fijado en validación.

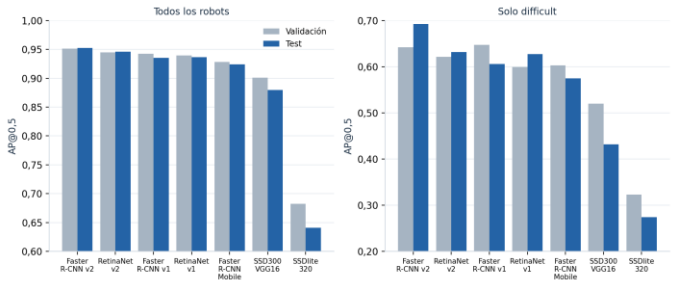


Figura 15. AP@0,5 en validación y en test. Interactiva en el Anexo A.

Faster R-CNN v2 con tasa 10^{-3} es el modelo final: en test obtiene prácticamente el mismo AP@0,5 que en validación, muy alto en los robots normales y bastante más bajo en los difficult (Tabla 18). Con el umbral de 0,95 detecta 1606 de los 1721 robots (apartado 10), y su F1 queda a menos de 0,3 pp del mejor posible en test. En general, test confirma validación: los modelos con ResNet-50 apenas cambian y las SSD pierden más.

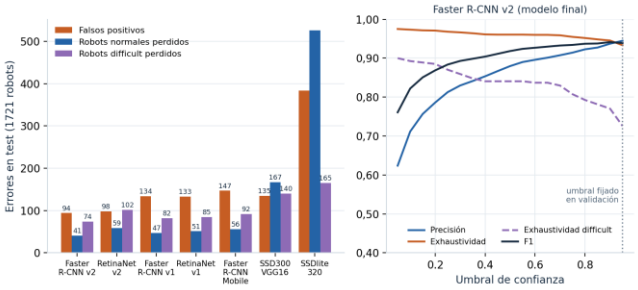


Figura 16. Errores en test con el umbral fijado en validación (izquierda) y curvas de umbral del modelo final (derecha). Interactivas en el Anexo A: izquierda y derecha.

Como se ve en la Figura 16, 74 de los 115 robots que no detecta el modelo final (64 %) son difficult, aunque representan solo el 16 % del total. Subir el umbral de 0,5 a 0,95 reduce los falsos positivos de 226 a 94, pero también baja la exhaustividad en difficult de 0,84 a 0,73.

Para la demo, Faster R-CNN MobileNet es una alternativa más rápida: pierde algo de precisión, pero tarda seis veces menos por imagen.

12. Análisis cualitativo

La Figura 17 muestra dos predicciones de Faster R-CNN v1 (lr 10^{-3}) sobre validación. En la primera hay un acierto completo, con tres detecciones correctas, incluida una con confianza 0,51 sobre un robot difficult. Las cajas verdes son aciertos y las rojas, detecciones sin pareja. En la segunda, de GR02_0001 de 2024, se confirmó que faltaban etiquetas (Figura 18, izquierda), por lo que algunos falsos positivos no son errores reales del detector, sino que el conjunto de validación no estaba bien etiquetado.



Figura 17. Predicciones de Faster R-CNN v1 (lr 10^{-3}) sobre validación, acierto en PIDS2025_GR01_0010/frame_1566 (TP 3, FP 0, FN 0) y el caso con más errores, PIDS2024_GR02_0001/frame_0492 (TP 2, FP 4, FN 0).

Al revisar las predicciones, se distinguen errores del detector y etiquetas incompletas. En la Figura 18, a la izquierda, aparecen cuatro robots, pero el XML solo incluye una caja, por lo que detectar los otros tres cuenta como FP. Las etiquetas se mantienen para conservar la comparación. Esto limita la interpretación de los 190 FP del modelo a 800 píxeles y los 324 del de 1024. Con el modelo final en test se revisaron uno a uno sus 94 FP: 59 son robots sin anotar (cortados por el borde, movidos, lejanos o tapados por otro robot), 31 son cajas mal ajustadas o repetidas sobre un robot anotado y solo 4 no son robots (tres sillas en penumbra y un coche). Con esos 59 robots anotados, la precisión pasaría de 0,945 a 0,979.



Figura 18. Izquierda: caso confirmado de anotación incompleta; la caja azul es la única del XML. Derecha: modelo a 1024 píxeles en PIDS2025_GR03_0011/frame_0600, con TP 4, FP 4 y FN 1 frente a las etiquetas.

Los fallos reales se concentran en tres casos. Primero, los objetos difficult, que acumulan 65 de los 83 FN de Faster R-CNN v1 (lr 10^{-3}) y 74 de los 115 del modelo final en test, aunque son solo el 16 % de los robots de validación y de test. Segundo, los robots lejanos, con un AP small de 0,12–0,19, que son difíciles de detectar y están poco representados: no hay cajas originales de menos de 30 píxeles de ancho. Tercero, algunas secuencias especialmente difíciles: en test, PIDS2025_GR02_0009 pierde 40 de 204 robots y PIDS2025_GR04_0000 pierde 10 de 21; en validación, GR04_0003 de 2024 pierde sus seis objetos difficult. También hay falsos positivos sobre una pierna y cajas solapadas alrededor de un robot (Figura 18, derecha), que no se explican por etiquetas omitidas.

12.1. Conclusiones

El modelo final elegido es Faster R-CNN v2 con lr 10^{-3}, cuyo checkpoint de la época 3 alcanza un AP@0,5 de 0,9523 en test (0,9658 en robots normales y 0,6921 en difficult) y un F1 de 0,9389 con el umbral 0,95 fijado en validación. Se prefiere a Faster R-CNN MobileNet, seis veces más rápido, por su mayor precisión, aunque habrá que comprobar en el robot si su latencia es adecuada.

La arquitectura influye más que la tasa de aprendizaje o el perfil de aumento. Los difficult siguen siendo el principal problema, las SSD tienen dificultades con los robots lejanos y Faster R-CNN sobreajusta pronto, por lo que el checkpoint se selecciona mediante validación.

13. Integración del detector en el robot

El Bloque 3 lleva el detector del Bloque 2 al RoboMaster EP, y añade un circuito en el que busca a los robots rivales con la cámara de la torreta, apunta y les dispara con el emisor de infrarrojos. El modelo es el elegido en el apartado 11.6: Faster R-CNN v2 con tasa de aprendizaje 10^{-3} y umbral de confianza 0,95.

El programa se ejecuta en un portátil unido al robot por wifi, con el SDK oficial [28], que solo funciona con Python 3.8. El PyTorch de esa versión no admite la RTX 5070 del portátil, así que el detector va en un proceso aparte, con Python 3.11 y PyTorch con CUDA 12.8, que carga el modelo y recibe cada imagen del programa del robot. Esa versión de PyTorch admite la RTX 5070 del portátil y la RTX 3060 de los ordenadores del laboratorio, así que la demo puede hacerse con cualquiera de los dos. Si la GPU falla, la imagen se analiza en la CPU como respaldo, sin parar la demo.

La latencia del detector condiciona todo el bloque: hay que repartir el tiempo entre girar, mirar y disparar. La configuración del circuito está separada del programa, y un robot simulado permite ejecutar la demo completa sin el RoboMaster, con imágenes del conjunto de test.

14. El circuito

El recorrido se define sobre un plano en metros, con el origen en la salida de nuestro robot. Cada tramo es un desplazamiento del chasis sin girarlo, ya que las ruedas mecanum permiten moverse hacia los lados y hacia atrás. Todos los tramos se midieron en el laboratorio. La Figura B3.1 muestra el recorrido y la Tabla B3.1, cada movimiento.

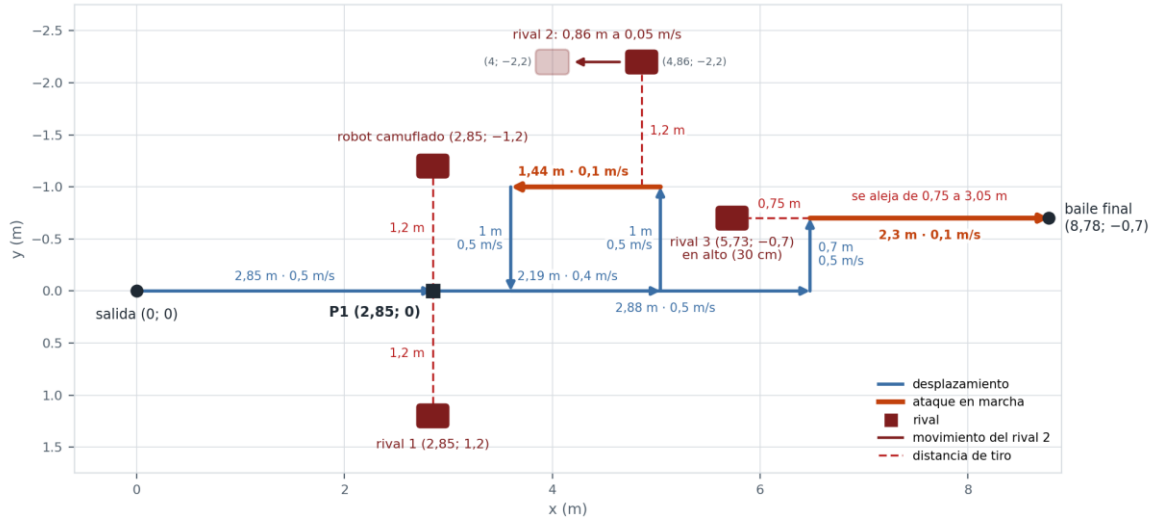


Figura B3.1. Recorrido de la demo con la longitud y la velocidad de cada tramo. Azul: desplazamientos; naranja: ataques en marcha; granate: rivales.

Tramo	Movimiento	Velocidad	Tiempo	Acción
P1	2,85 m adelante	0,5 m/s	unos 6 s (medido)	Suena un coche de Fórmula 1 ajustado a la recta
P1	Parada	—	—	Torreta a la derecha (90º): rival 1. Torreta a la izquierda (−90º): robot camuflado
R1	2,19 m adelante	0,4 m/s	unos 6 s	—
R1	1 m a la izquierda	0,5 m/s	unos 2,5 s	—
R1	1,44 m atrás	0,1 m/s	unos 14,5 s	Torreta a la izquierda: rival 2, que avanza 0,86 m en paralelo a 0,05 m/s, sin parar
R1	1 m a la derecha	0,5 m/s	unos 2,5 s	—
R2	2,88 m adelante	0,5 m/s	unos 6 s	—
R2	0,7 m a la izquierda	0,5 m/s	unos 2 s	—
R2	2,3 m adelante	0,1 m/s	unos 23 s	Torreta hacia atrás (180º): rival 3, escondido, sin parar
Final	Baile	—	lo que dure la canción	Música, torreta y chasis bailando y LED de colores

Tabla B3.1. Movimientos del circuito, medidos en el laboratorio.

15. Detección, puntería y disparo

En cada parada, la torreta gira a los ángulos previstos y analiza una imagen en cada uno. Con el ángulo de la torreta y la distancia entre la caja detectada y el centro de la imagen se calcula cuánto girar, en horizontal y en vertical.

15.1. Puntería por pasos

En el laboratorio el robot detectaba al rival con confianzas de 0,96 a 1,00, pero no llegaba a disparar: la torreta daba saltos y las imágenes de confirmación salían sin detecciones o con franjas verticales. El SDK guarda el vídeo en una cola de 64 imágenes que nadie lee mientras el modelo analiza o la torreta gira; al llenarse, las imágenes llegan antiguas o dañadas (Figura B3.2). Ahora un hilo lee el vídeo todo el tiempo y guarda solo la última imagen, y cada análisis espera a una imagen tomada con la torreta ya quieta.



Figura B3.2. Imágenes de la cámara del robot en el laboratorio el 4 y el 5 de noviembre.

Con cada imagen la torreta corrige lo que falta para centrar al rival, hasta cuatro veces, y dispara cuando el error es menor de 3° o del 30 % del ancho de la caja. Los giros grandes se dividen en tramos de 45° como máximo, porque el SDK solo garantiza unos 55° por orden.

15.2. Ataque en marcha

En dos tramos el robot dispara sin detenerse. Tres tareas se ejecutan a la vez: el chasis mantiene su velocidad, la torreta busca y sigue al rival, y una tercera lanza el aviso de voz y la ráfaga. Así el chasis no espera a la torreta ni a la ráfaga: sus órdenes caducan a los 0,5 s y su tarea las renueva sin pausa. La velocidad del retroceso sale de sumar los tiempos: girar la torreta 90° (3 s), encontrar y fijar al rival, el aviso (1,4 s) y la ráfaga (6,2 s). A 0,1 m/s, los 1,44 m duran unos 14,5 s, con margen para todo ello. Durante la ráfaga, la torreta sigue al rival con su velocidad angular estimada más una corrección limitada al 80 % del error, porque sigue girando mientras se analiza la imagen siguiente.

15.3. Disparo, sonido y luces

Cada ráfaga son 20 disparos de infrarrojos, uno cada 0,3 s. El enunciado pide 12 impactos, y los 8 de más cubren los que el rival no registre. El sonido de la SCAR apenas se oía, pues el clip dura 0,19 s, el altavoz corta el principio de cada audio y el SDK lo subía por FTP en cada disparo. Se subió el volumen unos 9 dB, se añadieron 0,3 s de silencio al principio y la ráfaga pasó a ser una sola pista de 6,2 s, con cada disparo lanzado en su instante para que suene a la vez.

Al detectar un rival suena un aviso de voz de 1,4 s y la ráfaga espera a que termine. En la primera recta suena un Fórmula 1 ajustado a sus 6,4 s, y al final el robot baila con Super Estrella. Los LED indican el estado: ámbar buscando, rojo parpadeando al disparar y verde tras eliminar a un rival.

16. Robot camuflado y robots aliados

La primera idea era un señuelo: algo con aspecto de robot al que nuestro RoboMaster no disparase, para enseñar que no ataca a todo lo que ve. Se probaron dos variantes, generadas con una IA de imágenes (Anexo F, Figura F.1): un RoboMaster disfrazado de detective, que el detector no debía reconocer, y cuatro robots aliados marcados en verde, a los que no debía disparar. Las pruebas dieron lo contrario, porque el detector los reconoce a todos como robots, y la idea se transformó: el disfrazado pasó a ser un objetivo más, el robot camuflado que la demo elimina en P1, y la identificación de aliados quedó como trabajo futuro, con una marca de color en lugar de una forma. Las imágenes se pasaron por el detector tal cual y dentro de un fotograma de la cámara, a distintos tamaños (Tabla B3.2).

Imagen	Confianza máxima	Veces que dispararía (de 5)	Verde dentro de la caja
RoboMaster disfrazado de detective	98,8 a 99,9 %	5	0,0 %
Aliado: RoboMaster S1 con bandera verde	99,8 a 99,9 %	5	5,7 %
Aliado: robot con orugas y cabeza redonda	99,8 a 99,9 %	5	11,0 %
Aliado: robot en dibujo	92,7 a 97,9 %	3	7,6 %
Aliado: robot blanco	95,4 a 98,9 %	5	12,5 %

Tabla B3.2. Respuesta del detector a imágenes de robots que no son RoboMasters reales.

El disfraz no engaña al detector, y tampoco una forma de robot distinta. Tapando partes del detective, la confianza apenas baja si se tapa una sola (ruedas, cañón o chasis); solo baja al 52 % si se tapan todas a la vez. El modelo ha aprendido el concepto de robot a partir del conjunto, no por una pieza concreta. En cambio, un círculo, una caja o una caja con ruedas dibujados no dan ninguna detección. La demo

aprovecha este resultado: en P1, el robot elimina al rival de la derecha y después al robot camuflado de la izquierda.

Para no disparar a un aliado hace falta una marca, no una forma. Los aliados tienen entre un 5,7 y un 12,5 % de verde intenso dentro de su caja y el detective un 0 %, así que una regla de «más de un 2 % de verde: aliado» los separa con margen. Es como en las competiciones de RoboMaster, donde los equipos se distinguen por el color de las luces. Se ha dejado como trabajo futuro, porque la luz del laboratorio y un fondo verde detrás de un enemigo podrían confundirla. El estudio completo está en el repositorio [21].

16.1. Prueba con el robot camuflado impreso

El robot camuflado se imprimió en papel para colocarlo en P1. Antes de llevarlo al laboratorio se comprobó con la webcam de un portátil y el mismo modelo de la demo: 20 imágenes, una por segundo, moviendo la hoja a distintas distancias. Se repitió con la misma imagen en la pantalla de una tableta (Tabla B3.3 y Anexo F, Figura F.2).

Soporte	Tamaño del robot en la imagen	Confianza máxima	¿Dispararía?
Papel	De un tercio a la mitad del alto	0,96 a 0,99	Sí
Papel	Un cuarto del alto	0,87 a 0,92	No
Papel	Lejos	0,53	No
Tableta	De un tercio a la mitad del alto	0,29 a 0,63	No
Tableta	Casi toda la imagen	0,55 a 0,95	Solo pegada a la cámara

Tabla B3.3. Robot camuflado impreso y en pantalla delante de la webcam del portátil, con el detector de la demo.

En papel, el detector reconoce al robot cuando ocupa una parte considerable de la imagen, y no dio falsos positivos sobre el fondo, donde la confianza máxima fue de 0,32. Con la pantalla los resultados empeoran mucho: su brillo hace que la cámara reduzca la exposición y oscurezca el resto de la escena, y los reflejos y el marco dificultan ver el contorno del robot. Por eso en la demo se usará impreso en papel mate A3, que evita esos problemas y permite que el robot se vea con tamaño suficiente desde P1.

Con la cámara del robot desde P1 dio de 0,72 a 0,97, casi siempre por debajo de 0,95, así que hacia ese lado basta un umbral de 0,7 si la caja no ocupa más de media imagen.

17. Control manual del robot

Además del circuito, se ha hecho una interfaz web para manejar el robot desde el portátil, con teclado y ratón o con un mando de PS4 que sirve para colocarlo, probar la cámara y el detector y hacer una demostración libre (Figura B3.3). Muestra el vídeo con las detecciones y permite mover el chasis y la torreta, disparar y lanzar bailes, con los mismos topes de velocidad que la demo. Si dejan de llegar órdenes durante 0,3 s, el robot se para. El detector va en el mismo proceso aparte con la GPU que la demo (apartado 13), lo que permite un disparo automático opcional.

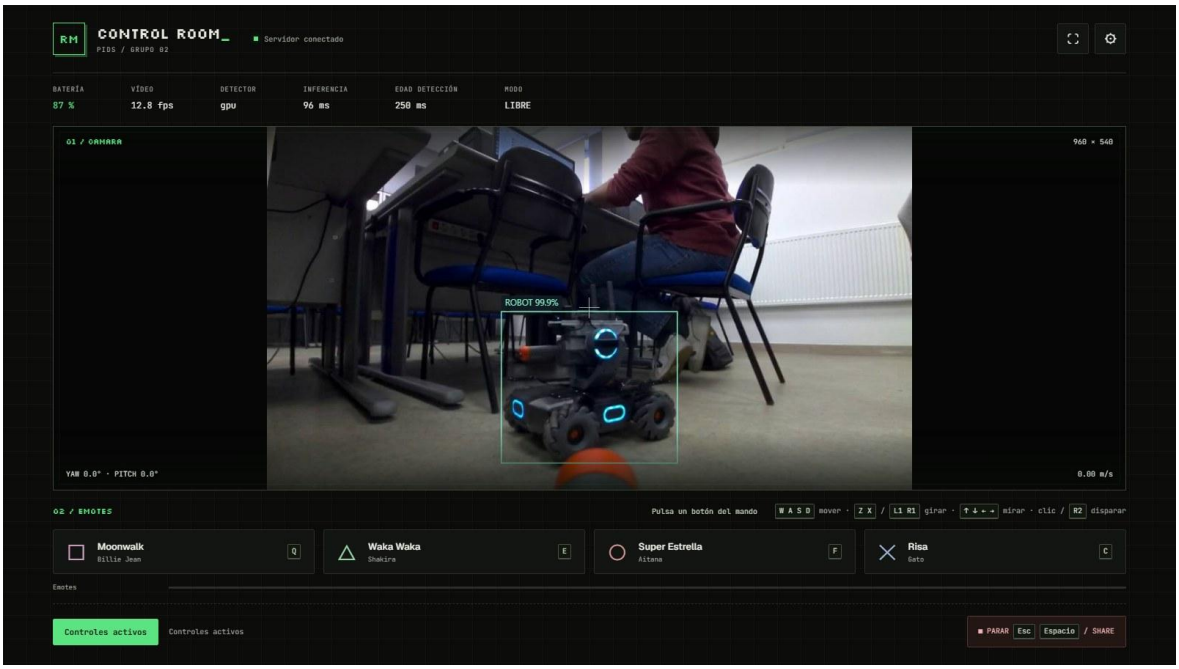


Figura B3.3. Interfaz del mando con una imagen de nuestra cámara en el laboratorio (robot simulado).

El 11 de noviembre, se probó con el robot real en el suelo del laboratorio, manejándolo con el mando de PS4 (Figura B3.4). El vídeo llegó a la interfaz a entre 8 y 16 imágenes por segundo y el detector marcó a los robots con confianzas de 0,97 a 1,00, también con varios a la vez y detrás de obstáculos. Los bailes se lanzan desde el mando y se avisan en pantalla.



Figura B3.4. Interfaz del mando con el robot real en el laboratorio el 11 de noviembre.

El límite es el hardware. El RoboMaster solo envía el vídeo por wifi y recibe órdenes, así que el detector se ejecuta en el portátil. En la demo cada imagen tardaba unos 108 ms, pero con la interfaz, que además muestra el vídeo, el disparo automático del 12/11 tardó de 130 a 570 ms por imagen. Para que vaya fluido haría falta una GPU de sobremesa o un modelo más ligero, y para no depender de la wifi, un ordenador con GPU a bordo del robot.

18. Pruebas y estado

Cada ejecución guarda las imágenes analizadas, con sus detecciones, tiempos y disparos, y una foto de la cámara cada 0,5 s. Las pruebas en el laboratorio del 4 al 11 de noviembre (Anexo G) sirvieron para corregir la cola de vídeo (apartado 15.1), alargar un 15 % los tramos tras el túnel y descartar sillas cercanas y robots del fondo por el alto de la caja y la distancia.

En la demo oficial en clase, el 12/11 (Figura B3.5), el robot cruzó el túnel de sillas, se paró en P1 y disparó 20 veces al rival 1 (confianza 1,00) y otras 20 al robot camuflado (0,99). En R1 disparó otras 20 al rival 2 mientras los dos se movían (1,00). La detección pequeña junto al rival 1, a unos 3 m y fuera del circuito, se descarta por la distancia.

El rival 3 se detectó con 1,00, pero no se eliminó (Anexo G, Figura G.2). Con la torreta girada 180°, la primera imagen por encima del umbral mostraba otro robot, al fondo y bajo una silla (0,99), y el rival 3 aún no se veía. La demo fijó ese robot, giró la torreta 40° hacia él y lanzó la ráfaga. Entonces apareció el rival 3, entre 30° y 36° a la derecha de la línea de tiro. Como el seguimiento toma por otro robot cualquier detección a más de 25° de donde espera al fijado, lo ignoró en las 43 imágenes siguientes, y la torreta siguió girando a la velocidad prevista en el plano, unos 5 °/s hacia atrás. Los 20 disparos le pasaron por la izquierda. El filtro de distancia de P1, aplicado también en marcha, habría descartado el robot del fondo.

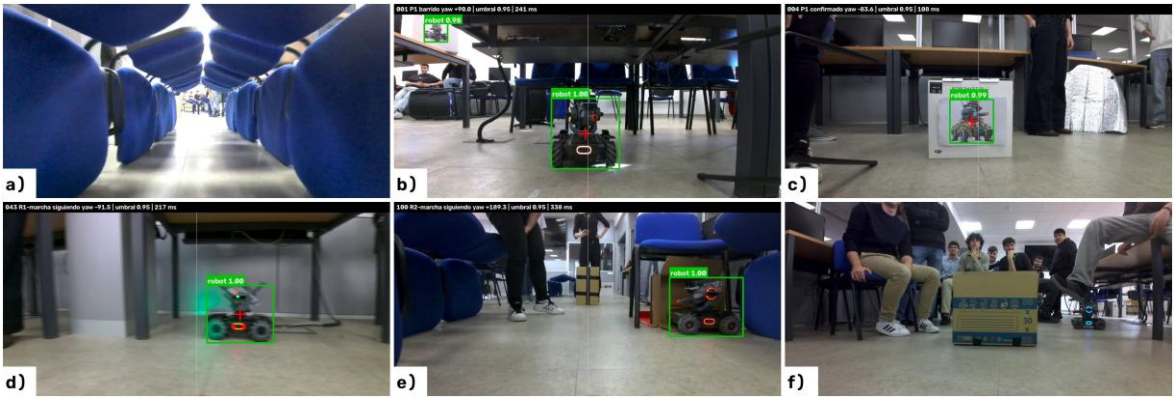


Figura B3.5. Cámara del robot en la demo oficial en clase, el 12 de noviembre.

Después, el profesor preparó un intento sorpresa con imágenes propias, robots tapados y un robot distinto (Figura B3.6). En P1 puso un robot tapado con una tela (0,85 a 0,91) y la foto de un robot entre humo (0,59); la demo no disparó a ninguno, porque quedaron por debajo del umbral. En la recta final colocó un robot con partes rojas, de un modelo que no está en nuestros datos. Lo detectó con 0,96 a 0,99 y le disparó, igual que al rival 2 (1,00). El detector generaliza a otros robots, pero un robot tapado o una imagen borrosa no alcanzan el umbral de 0,95, elegido para evitar falsos positivos.



Figura B3.6. Intento sorpresa del profesor en clase.

19. Conclusiones

Datos. El aumento de datos ha sido la mejora más importante, especialmente para los robots difficult. Su anotación también en validación y test ha permitido identificar los principales fallos del detector.

Modelo. La arquitectura tiene más impacto que los ajustes de entrenamiento. Faster R-CNN v2 es el modelo final, con un AP@0,5 de 0,95 en test. Sin embargo, sigue teniendo dificultades con los objetivos más complejos y todos los modelos presentan sobreajuste temprano.

Robot. La latencia del detector condiciona el funcionamiento: el robot debe repartir el tiempo entre movimiento, detección y disparo, utilizando imágenes recientes y nítidas. Además, el modelo reconoce qué es un robot, pero no distingue aliados de rivales, por lo que necesita una marca adicional para diferenciarlos. En la demo en clase detectó a los cuatro rivales y disparó a tres; al rival 3 la torreta no lo siguió porque había fijado otro robot del fondo. También detectó un robot distinto a los del entrenamiento, aunque no uno tapado con una tela.

Referencias

[1] labelImg. Herramienta de anotación PascalVOC. https://github.com/HumanSignal/labelImg

[2] Albumentations 1.4.18. Código de transformaciones. https://github.com/albumentations-team/albumentations/tree/1.4.18

[3] S. Ren et al. Faster R-CNN Towards Real-Time Object Detection with Region Proposal Networks. 2015. https://arxiv.org/abs/1506.01497

[4] torchvision 0.19. Faster R-CNN ResNet-50 FPN. https://docs.pytorch.org/vision/0.19/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html

[5] Repositorio del grupo. Datos, criterios, notebooks y resultados. https://github.com/javiersaguar/PIDS-Proyecto2

[6] Web del proyecto interactiva. https://javiersaguar.github.io/PIDS-Proyecto2/

[7] Notebook del EDA ejecutado (eda.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/eda.html

[8] Notebook del aumento de datos ejecutado (data_aug.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/data_aug.html

[9] T.-Y. Lin et al. Focal Loss for Dense Object Detection. 2017. https://arxiv.org/abs/1708.02002

[10] W. Liu et al. SSD: Single Shot MultiBox Detector. 2016. https://arxiv.org/abs/1512.02325

[11] K. He et al. Deep Residual Learning for Image Recognition. 2016. https://arxiv.org/abs/1512.03385

[12] T.-Y. Lin et al. Feature Pyramid Networks for Object Detection. 2017. https://arxiv.org/abs/1612.03144

[13] A. Howard et al. Searching for MobileNetV3. 2019. https://arxiv.org/abs/1905.02244

[14] K. Simonyan y A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. 2015. https://arxiv.org/abs/1409.1556

[15] T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. 2014. https://arxiv.org/abs/1405.0312

[16] Y. Li et al. Benchmarking Detection Transfer Learning with Vision Transformers. 2021. https://arxiv.org/abs/2111.11429

[17] M. Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018. https://arxiv.org/abs/1801.04381

[18] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. https://arxiv.org/abs/1912.01703

[19] DJI. RoboMaster EP. https://www.dji.com/robomaster-ep

[20] M. Everingham et al. The PASCAL Visual Object Classes (VOC) Challenge. International Journal of Computer Vision, 88, 303–338. 2010. https://doi.org/10.1007/s11263-009-0275-4

[21] Grupo 2. Robot camuflado y robots aliados: ¿a qué dispara el detector? 2026. https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/docs/senuelo_aliado.md

[22] H. Robbins y S. Monro. A Stochastic Approximation Method. The Annals of Mathematical Statistics, 22(3), 1951. https://doi.org/10.1214/aoms/1177729586

[23] I. Sutskever et al. On the importance of initialization and momentum in deep learning. ICML, 2013. https://proceedings.mlr.press/v28/sutskever13.html

[24] N. S. Keskar et al. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. 2017. https://arxiv.org/abs/1609.04836

[25] P. Goyal et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. 2017. https://arxiv.org/abs/1706.02677

[26] L. N. Smith. Cyclical Learning Rates for Training Neural Networks. 2017. https://arxiv.org/abs/1506.01186

[27] PyTorch. TorchVision Object Detection Finetuning Tutorial. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html

[28] DJI. RoboMaster SDK. Documentación. https://robomaster-dev.readthedocs.io/

Anexos

Anexo A. Figuras interactivas

La web del proyecto [6] reúne las versiones interactivas de las figuras de los dos bloques: permiten elegir vídeo, modelo, métrica o conjunto y consultar cada valor. La Tabla 19 relaciona cada figura o tabla del informe con su gráfica interactiva.

Figura o tabla	Gráfica interactiva	Enlace
Figura 6	Centros de las cajas de cada vídeo, todos y difficult	javiersaguar.github.io/PIDS-Proyecto2/#distribucion
Figura 6	Evolución de los centros en ventanas de 5 segundos	javiersaguar.github.io/PIDS-Proyecto2/#evolucion
Figura 7	Robots por fotograma en los diez vídeos	javiersaguar.github.io/PIDS-Proyecto2/#recuentos
Figura 11	Pérdidas y AP por época de los primeros experimentos	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-epocas
Tablas 14 y 15	Comparación de los primeros experimentos por métrica	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-comparacion
Apartado 11.2	AP de normales y difficult en cada experimento	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-grupos
Apartado 11.2	AP por secuencia de validación	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#titulo-secuencias
Figura 12	AP por época de cada arquitectura	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#arquitecturas
Figura 14	Pérdida, AP y tasa de cada entrenamiento, con SSD300 a 40 épocas	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#aprendizaje
Tabla 17	Efecto de la tasa de aprendizaje	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#tasa
Tabla 16	Tiempo de entrenamiento y época del mejor checkpoint	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#coste
Figura 13	Precisión frente a tiempo por imagen	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#latencia
Figura 15	AP@0,5 en validación y en test	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#validacion_test
Figura 16	Errores en test de cada modelo	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#errores

Figura o tabla	Gráfica interactiva	Enlace
Figura 16	Precisión, exhaustividad y F1 según el umbral de confianza	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#umbral
Tabla 18	AP por secuencia de test	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#secuencias
Apartado 11	Todas las medidas de los 13 entrenamientos en una matriz	javiersaguar.github.io/PIDS-Proyecto2/bloque2/#mapa

Tabla 19. Gráficas interactivas de la web del proyecto y su relación con el informe.

Anexo B. Transformaciones descartadas

Transformación	Motivo del descarte
Volteo vertical	El robot nunca está boca abajo y el suelo nunca aparece arriba.
Giros de 90°	La escena tiene una orientación fija; el robot no aparece de lado.
Perspectiva	En Albumentations 1.4.18 la caja devuelta no coincide con el robot (IoU de 0 en la prueba automática). Introduciría cajas falsas en el entrenamiento.
Recorte 640 × 480 redimensionado a 1280 × 720	Es el aumento de la asignatura. Pasa de 4:3 a 16:9 y ensancha los robots un 33 %; sobre nuestras imágenes pierde además el 14,1 % de los robots. Se sustituye por la escala sin deformar.
Desenfoque uniforme de 7 px al 50 %	El 27 % de los robots ya es difficult, casi siempre por movimiento. Desenfocar la mitad de las copias haría el entrenamiento más borroso que la realidad, y el desenfoque uniforme no se parece al de movimiento.
Intercambio de canales	Produce colores que la cámara nunca dará.
Niebla, lluvia, destellos y sombras en todas las imágenes	No son condiciones de la demo, que es en interior. Sombras y llovizna suaves se aplican solo a los vídeos exteriores (apartado 6.1).
Mosaico y copy-paste	Son útiles con muchos objetos pequeños; aquí hay 1,8 robots por imagen y la escala ya aporta robots más pequeños.

Tabla 20. Transformaciones descartadas y motivo.

Anexo C. Figuras complementarias

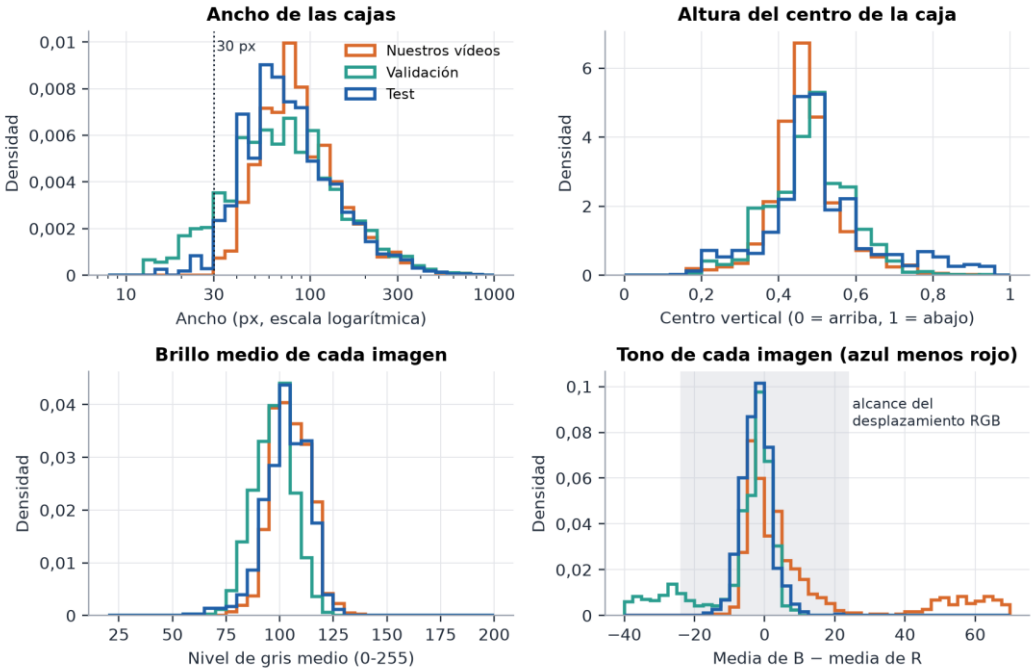


Figura 19. Distribución del ancho de las cajas, la altura de su centro, el brillo y el tono de cada imagen en nuestros vídeos, validación y test. La franja gris es el alcance del desplazamiento RGB del aumento (±12 por canal, hasta ±24 en B − R).



Figura 20. Una imagen de V06 y siete transformaciones del aumento por separado, con las cajas transformadas.

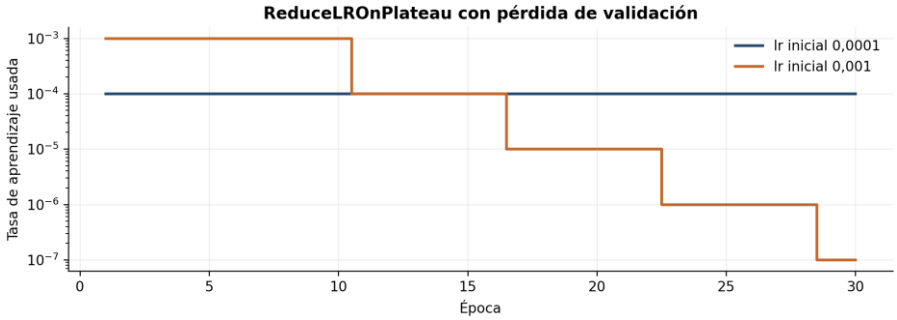


Figura 21. Tasa usada en cada época, y bajadas del programador.

La Figura 22 amplía el apartado 4.2 con análisis del notebook del EDA [7]. La nitidez (varianza del Laplaciano de la imagen en gris) depende del escenario: las medianas de los exteriores V04 y V09 son 328 y 278, y las de los interiores van de 43 a 130, porque con mucha luz la cámara usa exposiciones

cortas. Dentro de cada vídeo, cuanto menos nítida es una imagen, más cajas difficult tiene: el 55 % en el cuarto menos nítido (cuartiles de la nitidez de cada imagen respecto a la mediana de su vídeo, sobre las 767 imágenes con robots) frente al 11 % en el más nítido, y el 58 % de las imágenes con algún difficult frente al 21 %. Entre vídeos la relación es débil (V05 es de los menos nítidos y solo tiene un 10,6 % de difficult), por eso se mide dentro de cada vídeo. Como el criterio de anotación marca como difficult los robots movidos, esto es coherente con incluir desenfoque de movimiento en el aumento (apartado 5.3) y con que los difficult sean el principal error del Bloque 2. Por último, nunca hay más de tres robots por imagen y el 62 % tiene al menos dos.

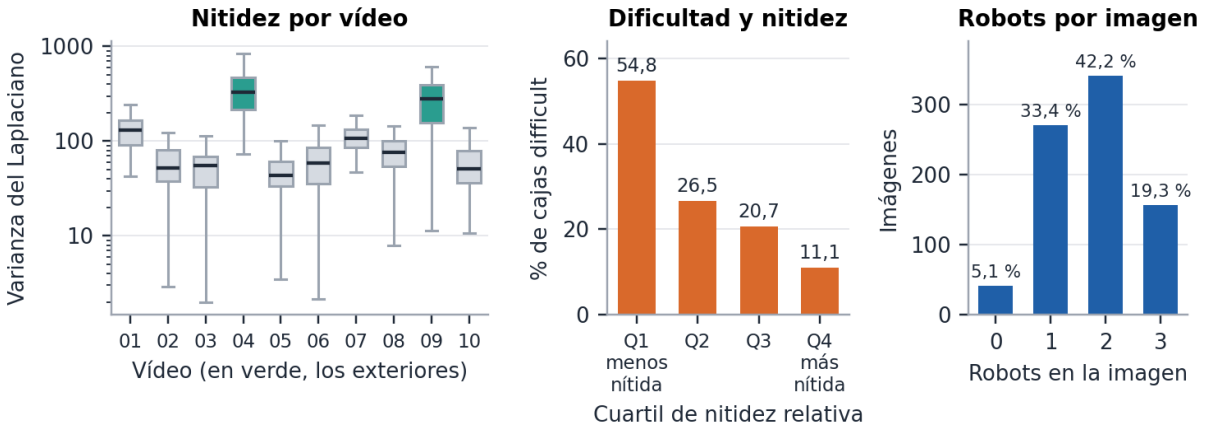


Figura 22. Izquierda: nitidez de las imágenes de cada vídeo, en escala logarítmica; en verde, los exteriores. Centro: porcentaje de cajas difficult según el cuartil de nitidez relativa de la imagen dentro de su vídeo. Derecha: robots por imagen, de las 808 imágenes.

Anexo D. Pruebas cortas: orden de los experimentos, lote y tasa de aprendizaje

Estas pruebas se hicieron al final para medir lo que no estaba en las fases anteriores. Todas se evalúan en validación, con la semilla 0 y la configuración de la Tabla 13 salvo lo que se indica, y entrenan solo con las 796 imágenes originales, sin aumento (100 actualizaciones por época con lote 8). El lote y la tasa se prueban con Faster R-CNN v1 durante tres épocas; la comparación de arquitecturas, durante seis. Cada resultado sale de una sola ejecución, por lo que diferencias de unas décimas no son concluyentes. RetinaNet y SSD300 mejoran hasta la última época y con ocho veces menos actualizaciones que los entrenamientos con aumento no han convergido, así que su posición en la Tabla 21 no es comparable con la de la Tabla 16.

Modelo	Mejor época (de 6)	AP@0,5 sin aumento	AP 0,50:0,95 sin aumento	AP@0,5 con aumento (12 épocas)	Minutos
Faster R-CNN v2	6	0,9382	0,6118	0,9513	18,8
Faster R-CNN v1	5	0,9347	0,5813	0,9426	13,2
RetinaNet v2	6	0,9188	0,5814	0,9447	12,6
Faster R-CNN MobileNet	5	0,9174	0,5768	0,9281	2,5
RetinaNet v1	6	0,8828	0,5727	0,9396	12,2
SSD300 VGG16	6	0,7754	0,4667	0,9008	2,3
SSDlite320	3	0,5975	0,3268	0,6823	0,8

Tabla 21. Siete arquitecturas entrenadas solo con las 796 imágenes originales (lr 10^{-3}, seis épocas, validación). Con mejor época 6, el AP@0,5 aún subía al terminar.

Lote	Pasos por época	AP@0,5 en las épocas 1 / 2 / 3	Mejor AP@0,5 (época)	AP 0,50:0,95	Memoria pico	Min
1	796	0,9329 / 0,9332 / 0,9280	0,9332 (2)	0,6084	1,3 GiB	7,2
2	398	0,9341 / 0,9358 / 0,9285	0,9358 (2)	0,5851	2,3 GiB	6,7
8	100	0,9128 / 0,9287 / 0,9326	0,9326 (3)	0,5886	8,0 GiB	6,6
16	—	Sin memoria (CUDA out of memory)	—	—	—	—
32	—	Sin memoria (CUDA out of memory)	—	—	—	—

Tabla 22. Efecto del tamaño del lote en Faster R-CNN v1 sin aumento (lr 10^{-3}, tres épocas, validación). El lote 8 son las tres primeras épocas de la prueba de seis. Los lotes 16 y 32 no caben en la GPU (11,6 GiB útiles): al fallar tenían asignados 10,48 y 9,47 GiB y pedían 1008 MiB más.

Tasa	AP@0,5 en las épocas 1 / 2 / 3	Mejor AP@0,5 (época)	Pérdida de entrenamiento (época 3)	Resultado
10^{-5}	0,0034 / 0,0089 / 0,0261	0,0261 (3)	0,461	Aprende demasiado despacio
10^{-3}	0,9128 / 0,9287 / 0,9326	0,9326 (3)	0,145	Referencia: sigue mejorando hasta la época 5
10^{-2}	0,9337 / 0,9353 / 0,9308	0,9353 (2)	0,107	Llega antes al máximo y baja después
10^{-1}	0,0000 / 0,0000 / 0,0000	0,0000 (1)	NaN	Diverge desde la primera época

Tabla 23. Efecto de la tasa de aprendizaje en Faster R-CNN v1 sin aumento (lote 8, tres épocas, validación).

Anexo E. Métricas de test

La Tabla 24 da, para el modelo final, las variantes de AP que calcula el notebook de evaluación. Las AP por tamaño siguen el criterio de COCO, según el área de la caja (menos de 32^2, de 32^2 a 96^2 y más de 96^2 píxeles), y la AR@100 es la exhaustividad media entre los umbrales de IoU de 0,50 a 0,95 con hasta 100 detecciones por imagen. La Tabla 25 muestra el compromiso del umbral de confianza (apartado 10): al subirlo de 0,05 a 0,95, los FP bajan de 1011 a 94, pero los FN suben de 43 a 115.

Métrica	Todos	Normales	Difficult
Robots anotados	1721	1451	270
Verdaderos positivos (TP)	1606	1410	196
Falsos negativos (FN)	115	41	74
Falsos positivos (FP)	94	–	–
Precisión	0,9447	–	–
Exhaustividad	0,9332	0,9717	0,7259
F1	0,9389	–	–
AP@0,5	0,9523	0,9658	0,6921
AP@0,75	0,6005	0,6386	0,3201
AP 0,50:0,95	0,5724	0,5942	0,3434
AP 0,50:0,95, robots pequeños	0,4743	0,6167	0,3336
AP 0,50:0,95, robots medianos	0,5286	0,5489	0,3306
AP 0,50:0,95, robots grandes	0,6053	0,6260	0,3644
Exhaustividad media (AR@100)	0,6558	0,6776	0,5389

Tabla 24. Métricas del modelo final en test (umbral 0,95). Las AP y la exhaustividad de cada columna se calculan solo con esos robots; los FP no pertenecen a ninguna clase, por lo que solo se dan en conjunto.

Umbral	TP	FP	FN	Precisión	Exhaust.	Exhaust. difficult	F1
0,05	1678	1011	43	0,6240	0,9750	0,9000	0,7610
0,50	1653	226	68	0,8797	0,9605	0,8407	0,9183
0,80	1638	137	83	0,9228	0,9518	0,7926	0,9371
0,90	1627	109	94	0,9372	0,9454	0,7704	0,9413
0,95	1606	94	115	0,9447	0,9332	0,7259	0,9389

Tabla 25. Modelo final en test con distintos umbrales de confianza (IoU ≥ 0,5).

Las Tablas 26 y 27 dan las mismas métricas para los siete modelos, cada uno evaluado una sola vez en test con el umbral de confianza fijado en validación (Tabla 18). En detección no hay verdaderos negativos: el fondo no se cuenta como objeto y el test no tiene imágenes sin robots, así que TN = 0 y

la exactitud (accuracy), (TP + TN)/(TP + TN + FP + FN), se reduce a TP/(TP + FP + FN). Un FP no corresponde a ningún robot, por lo que FP, precisión, F1 y exactitud solo se dan para todos los robots; TP, FN y exhaustividad se separan en normales y difficult. Faster R-CNN v2 es el mejor en AP@0,5, AP, precisión, exhaustividad, F1 y exactitud, y en todas las métricas de los difficult; Faster R-CNN v1 lo supera en AP@0,75 y las RetinaNet en AR@100. Las SSD son las que más pierden en robots pequeños y difficult.

Modelo	Grupo	AP	AP@0,5	AP@0,75	AP peq.	AP med.	AP gr.	AR@100
Faster R-CNN v2	Todos	0,5724	0,9523	0,6005	0,4743	0,5286	0,6053	0,6558
	Normales	0,5942	0,9658	0,6386	0,6167	0,5489	0,6260	0,6776
	Difficult	0,3434	0,6921	0,3201	0,3336	0,3306	0,3644	0,5389
RetinaNet v2	Todos	0,5609	0,9457	0,5845	0,2044	0,5018	0,6107	0,6575
	Normales	0,5844	0,9682	0,6185	0,2390	0,5243	0,6325	0,6800
	Difficult	0,3094	0,6313	0,2798	0,0860	0,2878	0,3497	0,5367
Faster R-CNN v1	Todos	0,5700	0,9355	0,6085	0,2051	0,5188	0,6105	0,6497
	Normales	0,5962	0,9583	0,6479	0,1592	0,5498	0,6307	0,6784
	Difficult	0,2979	0,6057	0,2453	0,1480	0,2674	0,3414	0,4952
RetinaNet v1	Todos	0,5711	0,9362	0,5986	0,1837	0,5054	0,6168	0,6586
	Normales	0,5954	0,9594	0,6376	0,1822	0,5281	0,6380	0,6827
	Difficult	0,3047	0,6273	0,2402	0,0906	0,2761	0,3469	0,5293
Faster R-CNN MobileNet	Todos	0,5335	0,9240	0,5272	0,0793	0,4667	0,5851	0,6080
	Normales	0,5590	0,9477	0,5638	0,1573	0,4973	0,6050	0,6387
	Difficult	0,2870	0,5744	0,2315	0,0000	0,2253	0,3614	0,4426
SSD300 VGG16	Todos	0,4788	0,8797	0,4360	0,0009	0,4118	0,5326	0,5757
	Normales	0,5095	0,9223	0,4699	0,0018	0,4463	0,5570	0,6061
	Difficult	0,2009	0,4316	0,1585	0,0000	0,1450	0,2705	0,4122
SSDlite320	Todos	0,3472	0,6412	0,3225	0,0127	0,1262	0,5029	0,5192
	Normales	0,3701	0,6750	0,3468	0,0244	0,1412	0,5256	0,5496
	Difficult	0,1297	0,2742	0,1087	0,0000	0,0136	0,2381	0,3559

Tabla 26. AP en test de los siete modelos por grupo de robots. AP es la AP 0,50:0,95, y AP peq., med. y gr., la misma AP para robots pequeños, medianos y grandes.

Modelo (umbral)	Grupo	TP	FP	FN	TN	Precisión	Exhaust.	F1	Exactitud
Faster R-CNN v2 (0,95)	Todos	1606	94	115	0	0,9447	0,9332	0,9389	0,8848
	Normales	1410	–	41	0	–	0,9717	–	–
	Difficult	196	–	74	0	–	0,7259	–	–
RetinaNet v2 (0,65)	Todos	1560	98	161	0	0,9409	0,9064	0,9234	0,8576
	Normales	1392	–	59	0	–	0,9593	–	–
	Difficult	168	–	102	0	–	0,6222	–	–
Faster R-CNN v1 (0,90)	Todos	1592	134	129	0	0,9224	0,9250	0,9237	0,8582
	Normales	1404	–	47	0	–	0,9676	–	–
	Difficult	188	–	82	0	–	0,6963	–	–
RetinaNet v1 (0,65)	Todos	1585	133	136	0	0,9226	0,9210	0,9218	0,8549
	Normales	1400	–	51	0	–	0,9649	–	–
	Difficult	185	–	85	0	–	0,6852	–	–
Faster R-CNN MobileNet (0,55)	Todos	1573	147	148	0	0,9145	0,9140	0,9143	0,8421
	Normales	1395	–	56	0	–	0,9614	–	–
	Difficult	178	–	92	0	–	0,6593	–	–
SSD300 VGG16 (0,25)	Todos	1414	135	307	0	0,9128	0,8216	0,8648	0,7619
	Normales	1284	–	167	0	–	0,8849	–	–
	Difficult	130	–	140	0	–	0,4815	–	–
SSDlite320 (0,20)	Todos	1030	384	691	0	0,7284	0,5985	0,6571	0,4893
	Normales	925	–	526	0	–	0,6375	–	–
	Difficult	105	–	165	0	–	0,3889	–	–

Tabla 27. Conteos en test de los siete modelos con IoU ≥ 0,5 y el umbral de confianza fijado en validación, entre paréntesis. TN = 0 y la exactitud es TP/(TP + FP + FN).

Anexo F. Robot camuflado y robots aliados

Imágenes del estudio del apartado 16. La Figura F.1 reúne los robots generados con IA. La Figura F.2 muestra capturas de la webcam del portátil con el detector de la demo: en verde, la caja que supera el umbral de 0,95 y con la que la demo dispararía; en naranja, las que no lo superan.



a) Detective b) Detective con bigote c) Aliado S1

d) Aliado con orugas e) Aliado en dibujo f) Aliado blanco

Figura F.1. Robots generados con IA: a) y b) RoboMaster disfrazado de detective, el robot camuflado de la demo; c) a f) robots aliados marcados en verde.

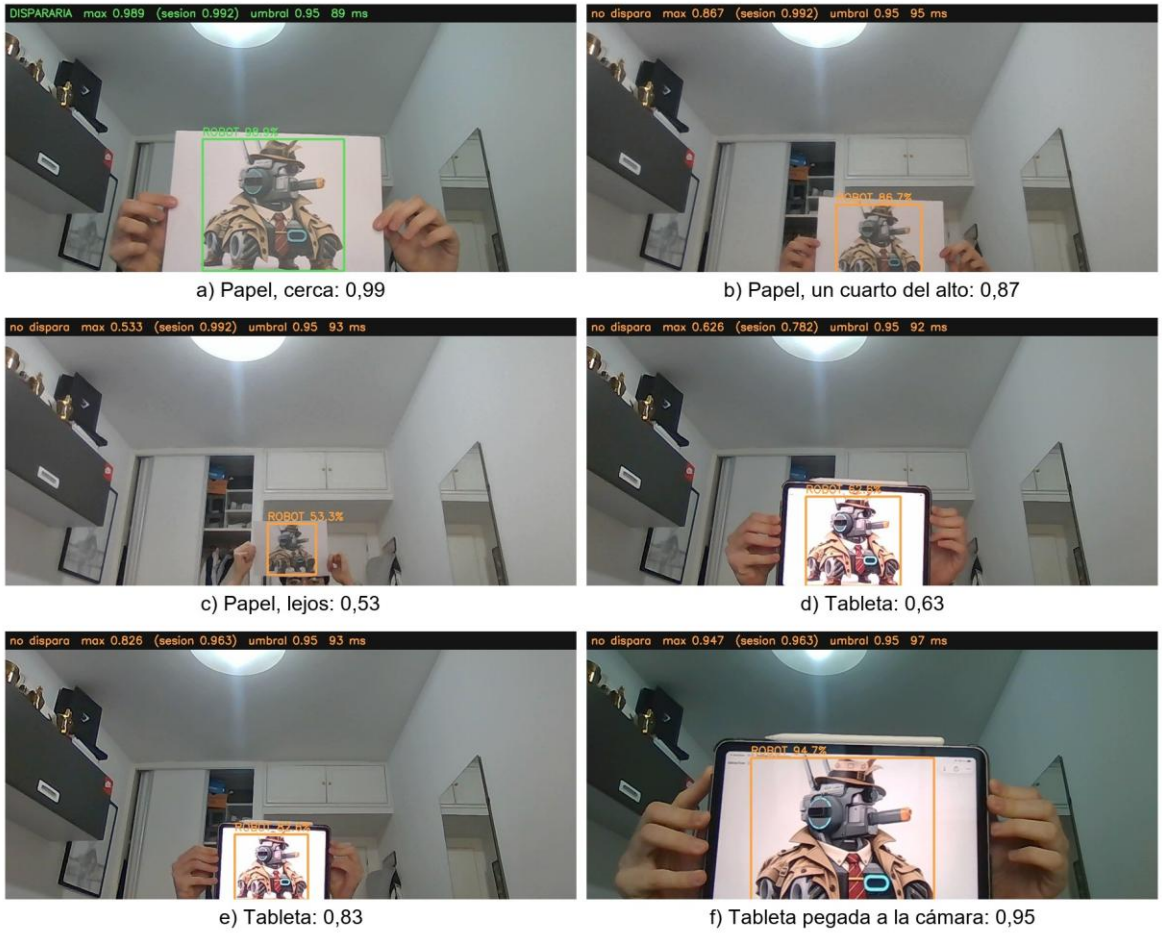


Figura F.2. Robot camuflado delante de la webcam del portátil, impreso en papel (a-c) y en la pantalla de una tableta (d-f), con la confianza máxima de cada imagen.

Anexo G. Desarrollo del circuito

Pruebas del circuito completo en el laboratorio el 11/11, la víspera de la demo en clase. La Figura G.1 muestra la imagen analizada al encontrar cada objetivo, con los rivales de entonces: la foto de la caja de un RoboMaster como rival 1 y el robot camuflado impreso sobre otra caja. La Figura G.2 muestra el ataque al rival 3 en la demo oficial del 12/11 (apartado 18).

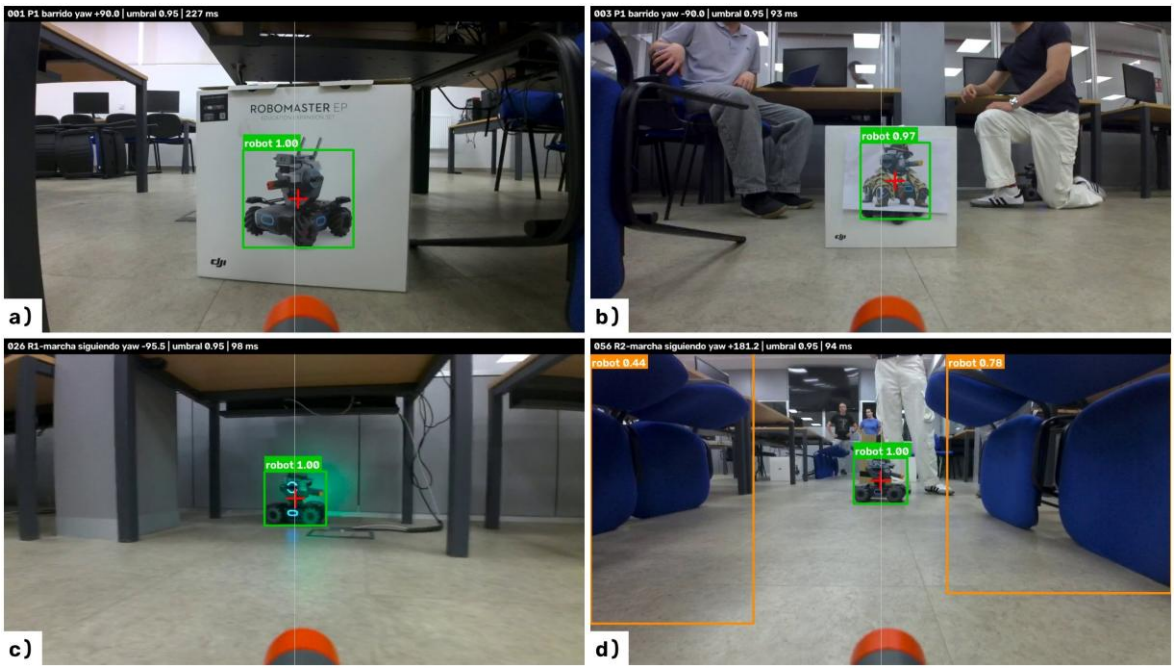


Figura G.1. Cámara del robot en las pruebas del 11 de noviembre, de dos ejecuciones: a) rival 1, la foto de la caja de un RoboMaster, a la derecha de P1 (confianza 1,00); b) robot camuflado impreso, a la izquierda de P1 (0,97); c) rival 2 en movimiento, durante el retroceso de R1 (1,00); d) rival 3 escondido entre las sillas, con la torreta girada 180º (1,00).

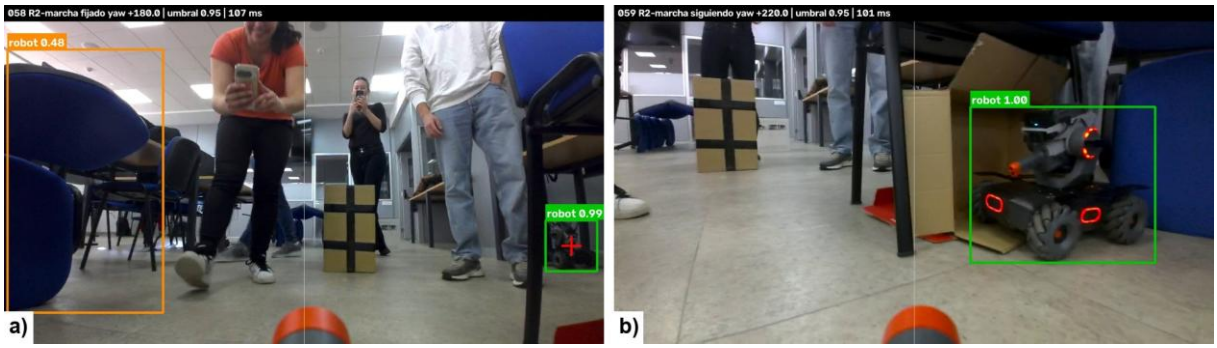


Figura G.2. Ataque al rival 3 en la demo oficial: a) con la torreta a 180°, la demo fija un robot del fondo, bajo una silla (0,99); b) tras girar a 220°, el rival 3 (1,00) queda a la derecha de la línea de tiro y el seguimiento lo descarta por estar a más de 25° del robot fijado.

Respuesta a la revisión anterior

La Tabla 28 recoge los comentarios de la revisión de la entrega 2 y dónde se ha incorporado cada corrección. Las de la entrega 1 se recogieron en la memoria de la entrega 2.

#	Comentario recibido	Sección	Cambio realizado
1	C2.1. Describir el sistema con un esquema de bloques de alto nivel (módulos y sus entradas y salidas) y explicar cómo cada módulo transforma sus datos.	7	La Figura 9 es ahora un esquema con solo los módulos y los datos que pasan entre ellos, y el texto explica qué hace cada módulo con sus entradas.
2	C2.2. Citar SGD.	7 y 9	Se citan SGD [22] y el momentum [23].
3	C2.3. Explicar la diferencia entre modelos de una y dos etapas.	8	Nuevo párrafo: propuestas y clasificación región a región frente a predicción directa sobre anclas, con las ventajas de cada uno.
4	C2.4. Profundizar en el batch size y usarlo para justificar el valor elegido.	9	Se explica por qué un lote pequeño da gradientes más variables, cuándo es bueno y cuándo malo [24, 25], y se justifica el lote de 8 con las pruebas de la Tabla 22.
5	C2.5. ¿Qué puede hacer una tasa de aprendizaje muy baja?	9	Convergencia lenta e infraajuste con un número fijo de épocas, con los ejemplos de 10^{-5} y 10^{-4}.
6	C2.6. Justificar momentum, weight decay y programador, o explicar una estrategia para elegirlos.	9	Qué hace cada uno, de dónde salen los valores [3, 27], por qué la paciencia de 5 épocas resultó larga y una estrategia de ajuste con la prueba de rango de la tasa [26].
7	C2.7. Añadir una imagen de la curva precisión-exhaustividad con resultados reales.	10	Nueva Figura 10: curvas en test del modelo final y de Faster R-CNN MobileNet.
8	C2.8. Explicar las resoluciones (800/1333) frente a los 1280 × 720 de grabación.	9 y 11.2	Se explica el reescalado con lado corto y lado largo máximos: 1280 × 720 entra a 1333 × 750 y, en el experimento de resolución, a 1820 × 1024.
9	¿Se etiquetaron como difficult las imágenes de validación y test?	7	Sí: se explica cómo se marcaron sus 525 difficult, que venían sin marcar.
10	Explicación deficiente de la FPN.	8	Se explica el recorrido de arriba abajo, las conexiones laterales y la elección del nivel según el tamaño del robot.
11	«Permite acelerar la convergencia del modelo».	8	Corregido.
12	¿Por qué microlotes de 4 y no de 6?	9	Los microlotes deben dividir a 8 para que todos los pasos tengan 8 imágenes; 4 es el mayor que cabe.
13	Usar notación científica.	9, 11 y anexos	Tasas, weight decay y mínimos del programador en notación científica.
14	¿«Puntos» = %?	10	Se definen los puntos porcentuales (pp) y se usan en el Bloque 2.
15	«Los experimentos de comparación entre modelos».	9	Corregido.
16	Opcional: nombres autoexplicativos para los experimentos y tendencias en vez de valores exactos.	11	Los experimentos de la Tabla 14 se nombran por lo que cambian y el apartado 11 compara tendencias; los valores quedan en las tablas.

Tabla 28. Trazabilidad de las correcciones respecto a la entrega 2.